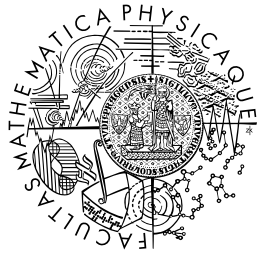
TODOs



FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Filip Bubák

Graphle: Graph-oriented file management system

Department of Software Engineering

Supervisor of the bachelor thesis: Ing. Pavel Koupil, Ph.D.

Study programme: Programming and Software Development

Prague 2025

The author declares that this bachelor thesis was carried out independently, and only with the cited sources, literature and other professional sources. It is understood that the work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In . . . . . date . . . .

Author's signature

I would like to thank my supervisor, Pavel Koupil, for his guidance and support during my work on this thesis.

Title: Graphle: Graph-oriented file management system

Author: Filip Bubák

Department: Department of Software Engineering

Supervisor: Ing. Pavel Koupil, Ph.D., Katedra softwarového inženýrství

Abstract: Operating systems organize files in directory trees, which forces each file into one primary location and makes it difficult to express multiple semantic associations. This thesis presents Graphle, a graph-oriented file management system that extends an existing filesystem with tags and typed relationships between files and folders. Files remain stored in their original directories, while semantic metadata is kept in a labeled property graph. The result of this thesis is a working application that supports graphical and command-based interaction, preserves compatibility with ordinary filesystem tools, and demonstrates the feasibility of graph-based file organization.

Keywords: file management, filesystem, labeled property graph, file tagging, semantic relationships, domain-specific language

Název práce: Graphle: Grafově orientovaný systém pro správu souborů

Autor: Filip Bubák

Katedra: Katedra softwarového inženýrství

Vedoucí bakalářské práce: Ing. Pavel Koupil, Ph.D., Katedra softwarového inženýrství

Abstrakt: Operační systémy organizují soubory do adresářových stromů, což každý soubor omezuje na jedno primární umístění a ztěžuje vyjádření více sémantických vztahů. Tato práce představuje Graphle, grafově orientovaný systém pro správu souborů, který rozšiřuje stávající souborový systém o značky a typované vztahy mezi soubory a složkami. Soubory zůstávají uloženy ve svých původních adresářích, zatímco sémantická metadata jsou uchovávána v grafu vlastností s popisky. Výsledkem této práce je funkční aplikace, která podporuje grafickou i příkazovou interakci, zachovává kompatibilitu s běžnými nástroji pro práci se souborovým systémem a dokazuje proveditelnost grafové organizace souborů.

Klíčová slova: správa souborů, souborový systém, graf vlastností s popisky, značkování souborů, sémantické vztahy, doménově specifický jazyk

Contents

Section 1

Introduction

Operating systems organize files as a strict tree of directories, where every file resides at exactly one location. This imposes a single, rigid classification onto data where the user has to choose the most appropriate aspect of the file and place it into a corresponding folder. However, this is often difficult. A photograph may belong at once to a holiday, a person, and a specific project. A tree cannot express such many-to-many associations, which makes organizing files harder than it should be. Some tools try to bridge this gap, but each does so only in its own limited way. They do not let users freely connect arbitrary files with typed relationships (a logical connection between entities describing how they are conceptually related) and tags across the whole filesystem in an easily queryable way.

People associate one thing with many areas at once, which a graph expresses more naturally than a tree. If we treat files and folders as nodes joined by typed relationships and annotated with tags (a category of an entity), a file still resides in a single location, yet it can be referenced from anywhere in the graph.

The objective of this thesis is to design and implement Graphle, a graph-oriented file management system that lets users organize their files the way they think. Graphle represents files and folders as nodes in a labeled property graph connected by arbitrary, user-defined relationships and tags, while remaining backward-compatible with the existing filesystem (the operating system structure that stores files and directories and controls access to them). It enables two modes of interaction: either a graphical client or a custom query DSL (domain-specific language). The thesis delivers a working client and backend, a graph data model, and a query language for files, while the system remains backward-compatible with the directory hierarchy.

Outline. The remainder of this thesis is organized into six chapters. Section 2 establishes the functional and qualitative requirements, user roles, security model, and use cases. Section 3 surveys existing file managers, knowledge-management applications, and navigation utilities, and compares them with Graphle. Section 4 covers the graph data model, architecture, module decomposition, technologies, and mockups of the user interface. Section 5 documents the backend and client implementations, the API, and the key algorithms. Section 6 covers installation, the user interface, and a tutorial for both the graphical and DSL workflows. Finally, Section 7 summarizes the contributions and outlines future work.

Section 2

Analysis

A standard filesystem organizes files in a strict hierarchy of folders. This forces users to pick a single location for each file and makes it difficult to express a relationship that spans multiple topics or projects. This section provides the analytical documentation for an application designed to extend existing filesystems with relationships and tag support, enabling graph-based navigation without replacing or breaking the underlying directory structure. It targets both regular users who benefit from a more expressive GUI file browser and power users such as developers and system administrators who can automate operations through the provided DSL. The section defines functional and qualitative requirements, including semantic and hierarchical connection (an edge between entities in a database) support, interaction with the system, and its qualities. It explains how the system handles user accounts and their file permissions. The last subsection shows possible interactions through use cases, including creating and deleting semantic information, browsing through the filesystem, and using the DSL.

2.1 Functional and Qualitative Requirements

The requirements were defined based on an analysis of existing tools and their limitations. Existing filesystem tools and productivity applications were examined to identify potential enhancements that can be made to them. The existing solutions will be discussed further in the Current landscape section. The thesis was not commissioned by an external client. Instead, the author acted as the primary stakeholder for the project. The requirements were derived from the author's experience with the problem domain and adjusted based on supervisor feedback and informal interviews with potential users. The main goal of this project is to extend a standard filesystem to support faster traversal by interpreting relationships as an LPG (Labeled Property Graph).

2.1.1 Functional Requirements

This list serves as a feature list that this application should fulfill in order to be considered complete. Each requirement has a priority, an acceptance criterion, and a link to the use case or chapter that explains it further.

ID	Priority	Requirement	Acceptance criterion	See
F1	Must	Backwards compatibility	Graphle works on top of the existing filesystem without copying file contents, and it exposes parent and child directory entries as graph neighbors.	Section 2.4.1
F2	Must	Relationships between files	The user can create a typed relationship between two files or folders, see it from the	Section 2.4.7, Section 2.4.8

ID	Priority	Requirement	Acceptance criterion	See
			source file, traverse it, and remove it.	
F3	Must	File tags	The user can add a tag to a file or folder, find files by that tag, and remove it.	Section 2.4.9, Section 2.4.10
F4	Must	File manipulation	The user can create, open, delete, and move files through Graphle, and the change is reflected in the underlying filesystem.	Section 2.4.2, Section 2.4.3, Section 2.4.5, Section 2.4.4
F5	Must	Folders work as files	A folder can be browsed, tagged, connected by relationships, moved, and deleted through the same workflows as a file.	Section 2.4.1, Section 2.4.9, Section 2.4.7, Section 2.4.4, Section 2.4.5
F6	Must	GUI file browser	The user can open the GUI, navigate through neighbors, and select a file or folder.	Section 2.4.1
F7	Must	Custom DSL	The user can execute DSL commands for search and graph operations, and the system returns either matching objects or a clear error.	Section 2.4.11
F8	Should	Filename autocompletion	While the user types a DSL command, the system displays filename suggestions based on known or recently visited paths.	Section 2.4.12
F9	Must	Lazy updates	Files are loaded from the filesystem when they are visited or queried, and files deleted outside Graphle are removed from the graph by later navigation or background cleanup.	Section 2.4.1
F10	Must	Cross-platform support	The backend and GUI can be built and run on macOS and Linux using the documented installation steps.	Section 6

Windows is explicitly not supported, as NTFS assigns each volume an independent drive letter rather than providing a single root hierarchy. Without a common root, files on different drives cannot be connected in the graph.

2.1.2 Qualitative Requirements

Qualitative requirements define constraints on how the functionality should behave.

ID	Priority	Requirement	Acceptance criterion	See
Q1.1	Must	Usability - Interaction modes	Finding files, editing tags, and editing relationships are available through both the GUI and the DSL.	Section 2.4.1, Section 2.4.11
Q1.2	Should	Usability - Theming and replaceable client	The default GUI supports multiple visual themes, and another client can use the public API without direct database access.	Section 4, Section 5
Q1.3	Should	Usability - Remote access	A user can connect to a remote instance of `GraphleManager` via SSH port forwarding and browse the filesystem of the machine where the backend runs.	Section 2.4.6
Q2.1	Must	Performance - Autocomplete latency	For an established GUI connection and warm caches, filename autocomplete returns suggestions within 250 ms.	Section 2.4.12
Q3.1	Should	Reliability - Autocomplete retries	The GUI can reconnect to the autocomplete WebSocket (a protocol for a persistent bidirectional connection between a client and a server) without restarting the application.	Section 2.4.12
Q3.2	Must	Reliability - Component isolation	If an auxiliary component such as Valkey or a GUI client fails, core file, tag, and relationship operations continue to work, possibly with degraded autocomplete.	Section 2.4.1, Section 2.4.9, Section 2.4.7
Q4.1	Should	Extensibility - New commands	Adding a simple DSL command can be done by extending the parser or command dispatch path without changing unrelated storage or transport layers.	Section 2.4.11

2.2 User roles

The application targets both regular users and power users. The user can choose which functionality they want to use and does not need to adjust to a more powerful way of traversing files all at once.

2.2.1 Regular User

A regular user is anyone who wants to organize their files beyond a plain folder hierarchy. They interact with the application through the GUI, browsing the filesystem, assigning tags to files, and navigating relationships between them. No special technical knowledge is required beyond basic computer literacy. This user benefits from a gradual transition to a more expressive way of organizing and discovering files, as the application layers on top of the existing filesystem without requiring any upfront changes.

2.2.2 Power User

A power user is a technically proficient user, most often a developer or system administrator, who wants to automate or script filesystem operations. They interact with the application primarily through the custom DSL, which allows them to perform batch operations and more advanced queries that would require a lot of effort using a GUI. This user is familiar with command line tools and basic scripting. Power users benefit from being able to integrate the Graphle DSL into their existing and new scripts.

2.3 Security

Since the application operates directly on the user's filesystem and can be accessed remotely through SSH port forwarding, security must be considered. The current implementation does not provide application-level authentication. Instead, it relies on the operating system account under which `GraphleManager` runs, filesystem permissions, and the access control already provided by SSH. This is a deliberate limitation of the bachelor's thesis implementation rather than a complete security model. Multi-user support and authorization for remote clients are left for future work.

2.3.1 Accounts

The system will not have the ability to create custom users, as operating systems already provide this functionality. The user account used is the account of the user who launched the application. Therefore, to create a new user, the user creates a new user account in the operating system. This has the benefit that the system itself does not need to concern itself with storing user credentials, which could be a significant security liability if the system were exposed to the internet. Adding a separate credential system and session handling would duplicate mechanisms already provided by the operating system and SSH while increasing the amount of security-critical code needed to be maintained by Graphle. Instead, users are advised to set up SSH port forwarding [1], as described in the deployment documentation in Section 6.2.1. This is the method that many other applications, such as DBeaver [2], use for safer access. The downside of this approach is that Graphle cannot distinguish between multiple remote users. These limitations are acceptable for the current version, and are left as an extension for future work.

2.3.2 File permissions

The application inherits all the file permissions from the user who launched the system. To restrict a user's access, create a new user account for them and assign the necessary permissions.

2.4 Use Cases

This section outlines the use cases that describe how the user interacts with the application. Use cases cover a variety of functions, including file manipulation, creating and deleting semantic information, browsing through the filesystem, and using the DSL.

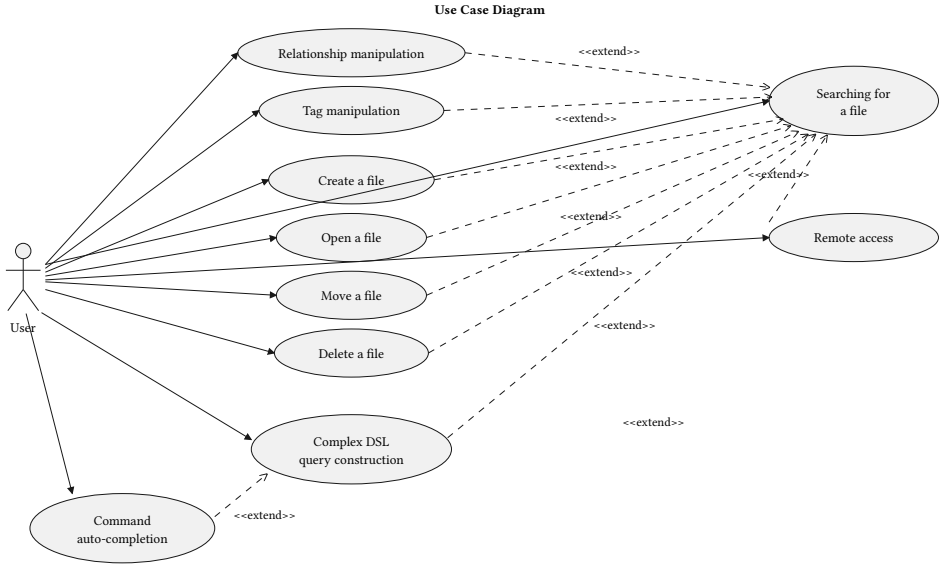


Figure 1: Use case diagram

The Figure 1 shows the overall picture of use cases and how they relate to each other.

2.4.1 Search for a file

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The user wants to find a file

Flow

1. The user opens the file browser
2. The file browser opens the user-defined home location
3. The application shows possible neighbor (an entity connected via a direct connection) entities from the current location
4. The user clicks on the neighbor they want to visit
5. If the file was not located, then go back to step 3
6. The system displays the selected file or folder

Alternative flow

- 4a) The user uses the integrated DSL for search

Postconditions

- The user accessed the desired file

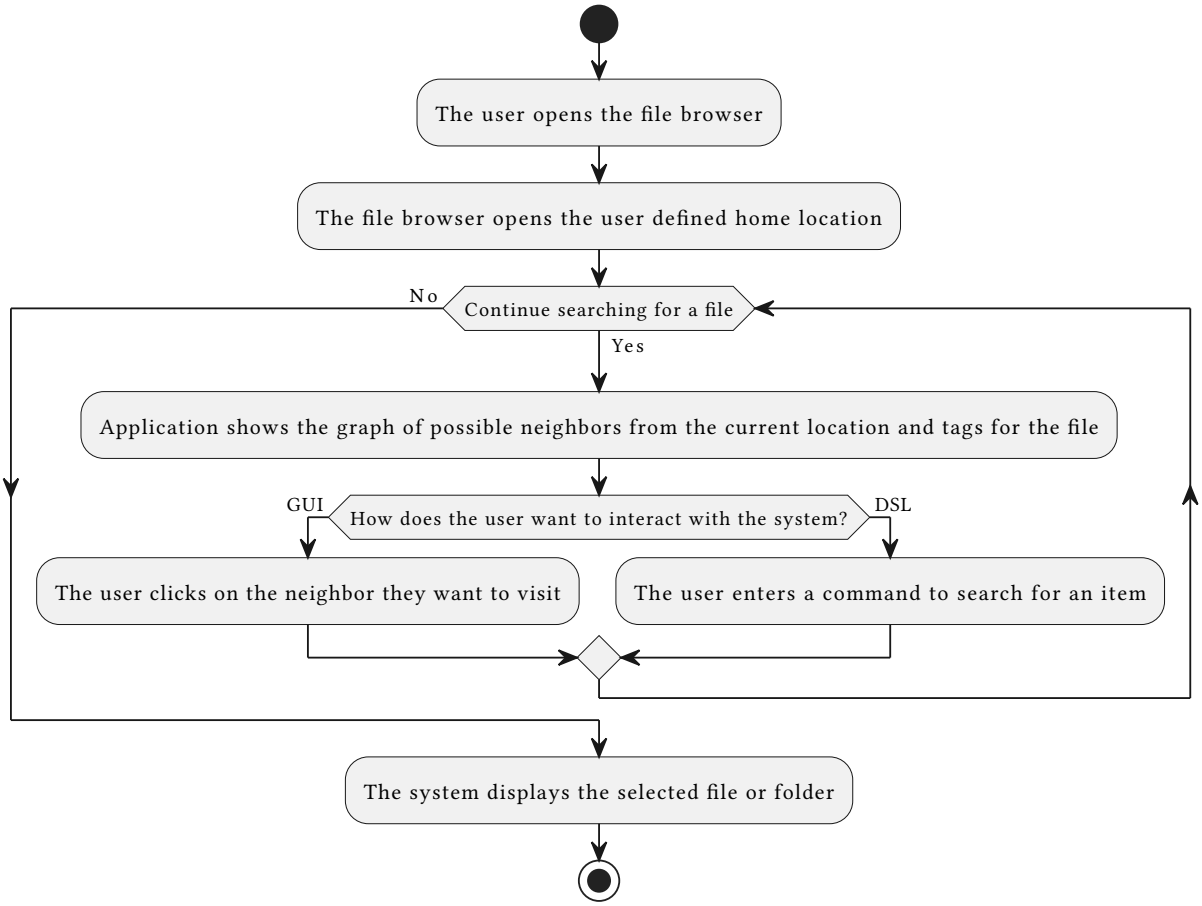


Figure 2: Search for a file activity diagram

2.4.2 Create a file

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The user wants to create a file in a known parent folder

Flow

1. The user searches for the parent folder
2. The user opens the context menu on the parent folder
3. The user selects the operation “Add file”
4. The system displays a dialog where the user can enter the new name
5. The user enters the name and submits the dialog
6. The system creates the file in the selected parent folder
7. The system shows the new file among the parent’s neighbors

Alternative flow

- 1a) The user issues a DSL command to create the file

- 5a) The user cancels the operation
- 6a) If the file cannot be created, the system shows an error and leaves the filesystem unchanged

Postconditions

- The new file exists in the underlying filesystem
- The user can open, tag, move, delete, or relate the new file through Graphle

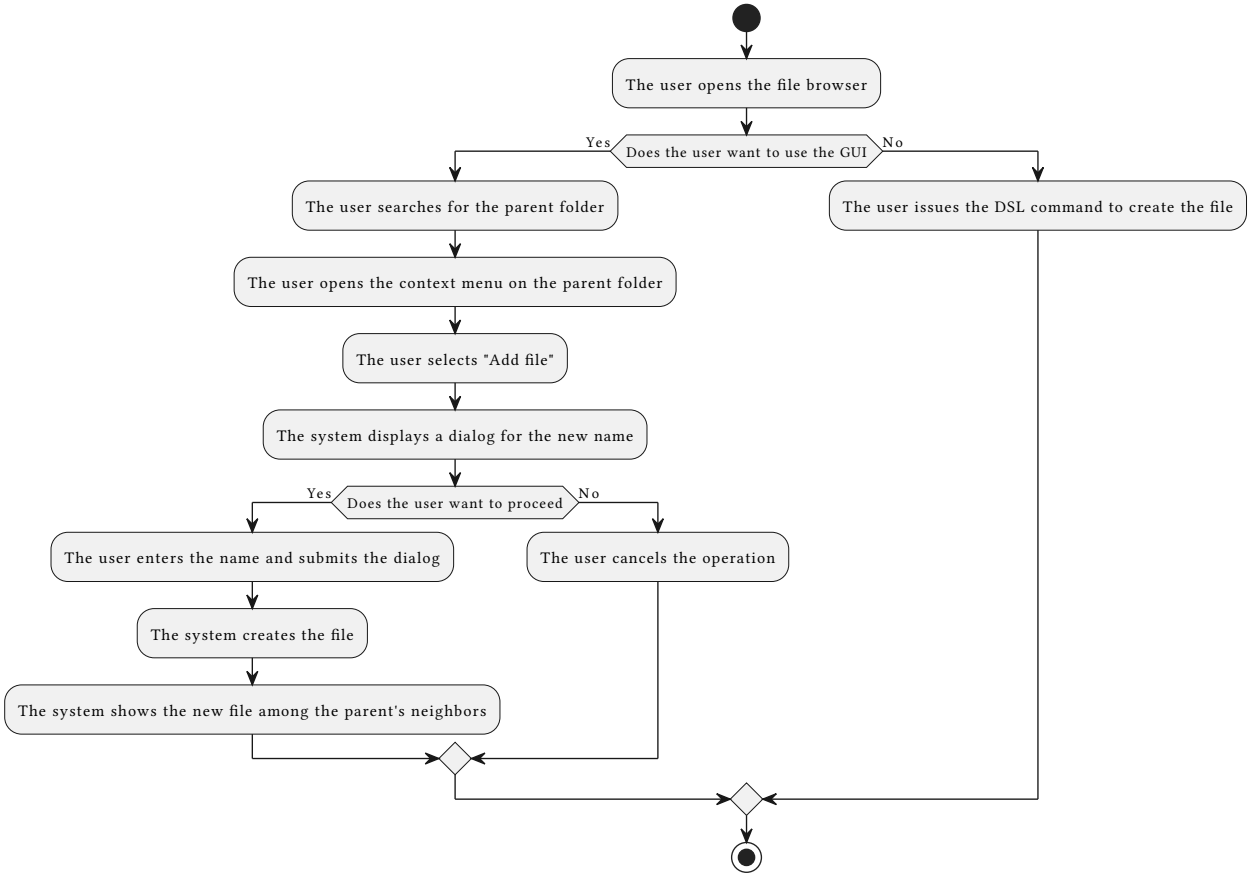


Figure 3: Create a file activity diagram

2.4.3 Open a file

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The user wants to open a file or folder

Flow

1. The user searches for the file or folder
2. The user opens the context menu for the selected item
3. The user selects the operation “Open”
4. The system opens the file or folder with the default application

Alternative flow

- 4a) If the backend runs on a remote machine, the GUI downloads the file first and then opens the local copy
- 4b) If the selected item is a remote folder, the operation is not offered and the user can browse the folder in Graphle instead
- 4c) If the file cannot be opened, the system shows an error and leaves the filesystem unchanged

Postconditions

- The selected file or folder is opened outside Graphle, or the user receives an error explaining why it could not be opened

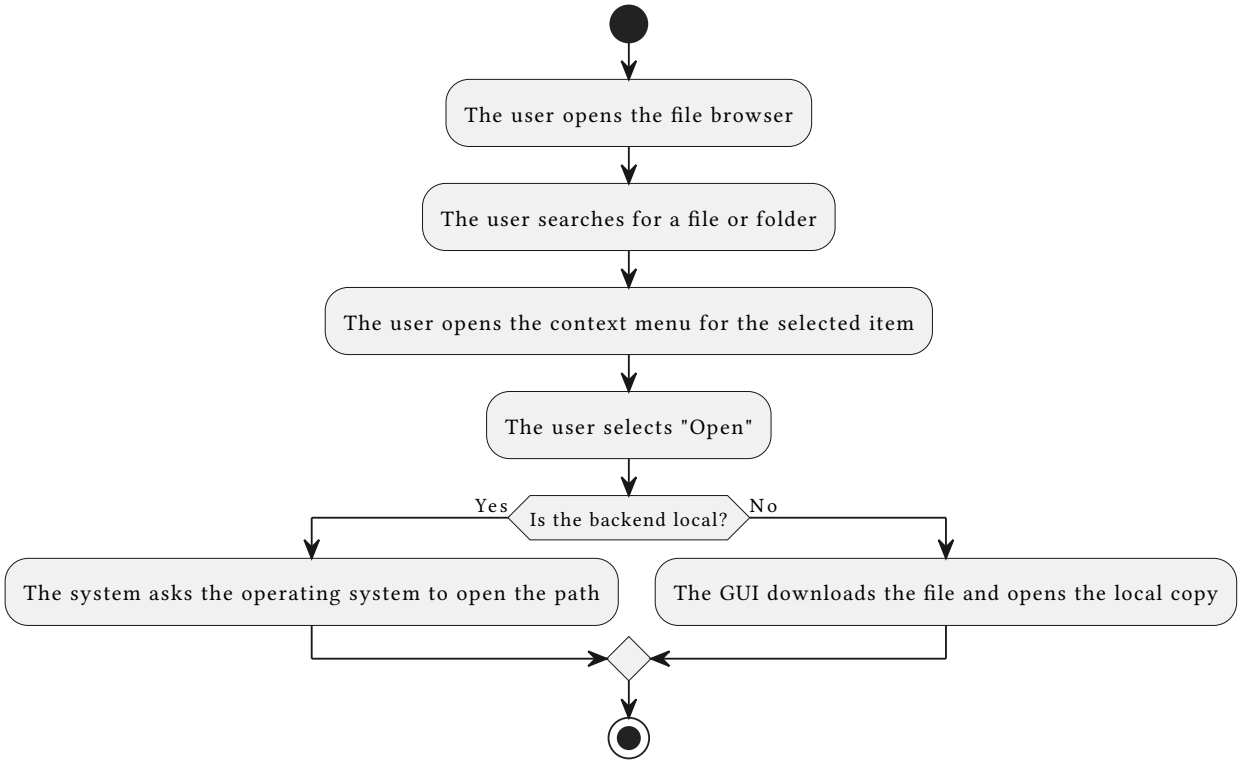


Figure 4: Open a file activity diagram

2.4.4 Move a file

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The user wants to move a file or folder to another location

Flow

1. The user searches for the file or folder
2. The user opens the context menu for the selected item
3. The user selects the operation “Move”
4. The system displays a dialog where the user can enter the destination
5. The user enters the destination and submits the dialog
6. The system moves the file or folder in the underlying filesystem

7. The system updates the stored graph location for the moved item

Alternative flow

- 1a) The user issues a DSL command to move the file
- 5a) The user cancels the operation
- 6a) If the destination is invalid or the move fails, the system shows an error and keeps the original location

Postconditions

- The file or folder exists at the destination path
- Tags and relationships for the moved item remain associated with its new location

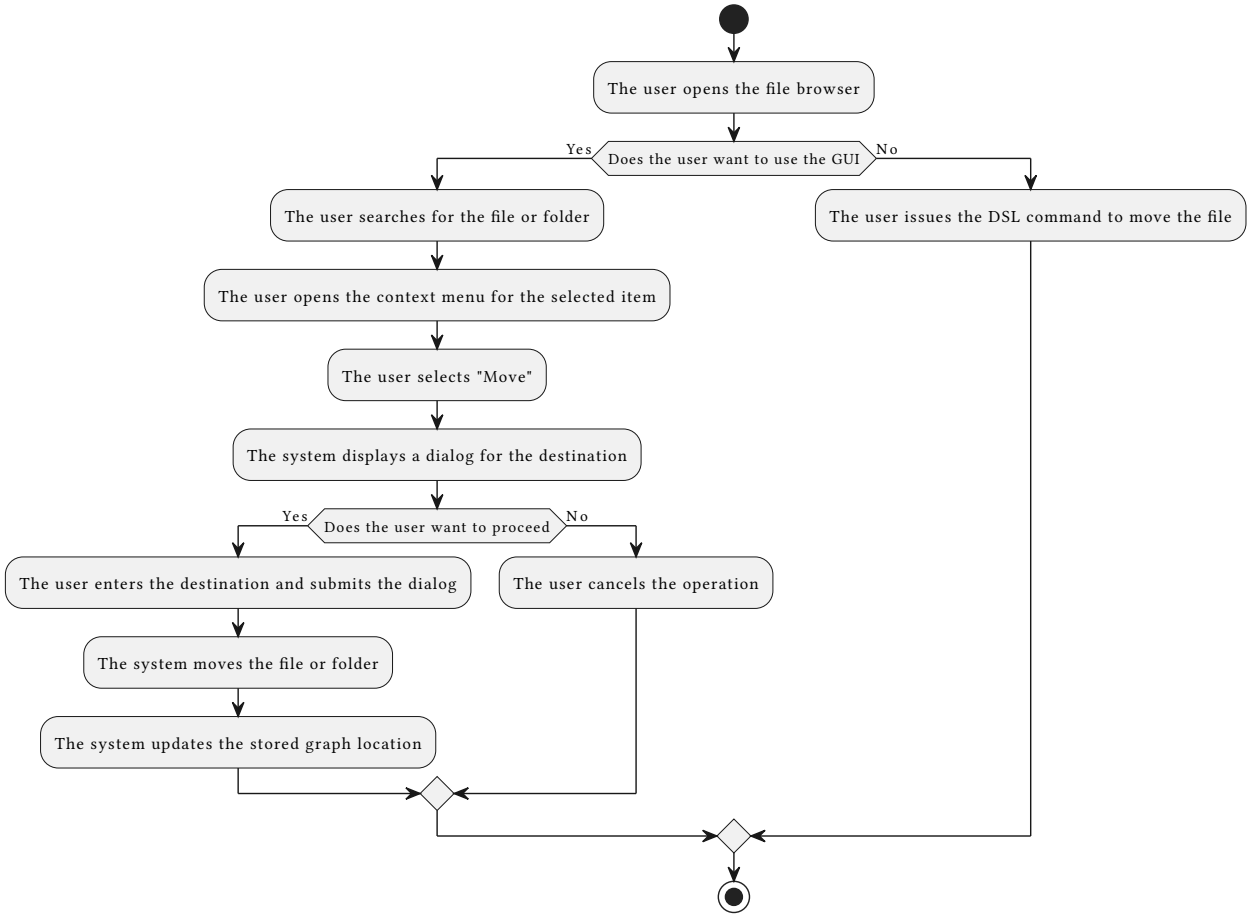


Figure 5: Move a file activity diagram

2.4.5 Delete a file

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The user wants to delete a file or folder

Flow

1. The user searches for the file or folder
2. The user opens the context menu for the selected item

3. The user selects the operation “Delete permanently”
4. The system displays a confirmation dialog
5. The user confirms the deletion
6. The system deletes the file or folder from the underlying filesystem
7. The system removes the deleted item and its metadata from the graph

Alternative flow

- 1a) The user issues a DSL command to delete the file
- 5a) The user cancels the operation
- 6a) If the file cannot be deleted, the system shows an error and keeps the graph metadata unchanged

Postconditions

- The file or folder no longer exists in the underlying filesystem
- The deleted item is no longer shown as a graph entity in Graphle

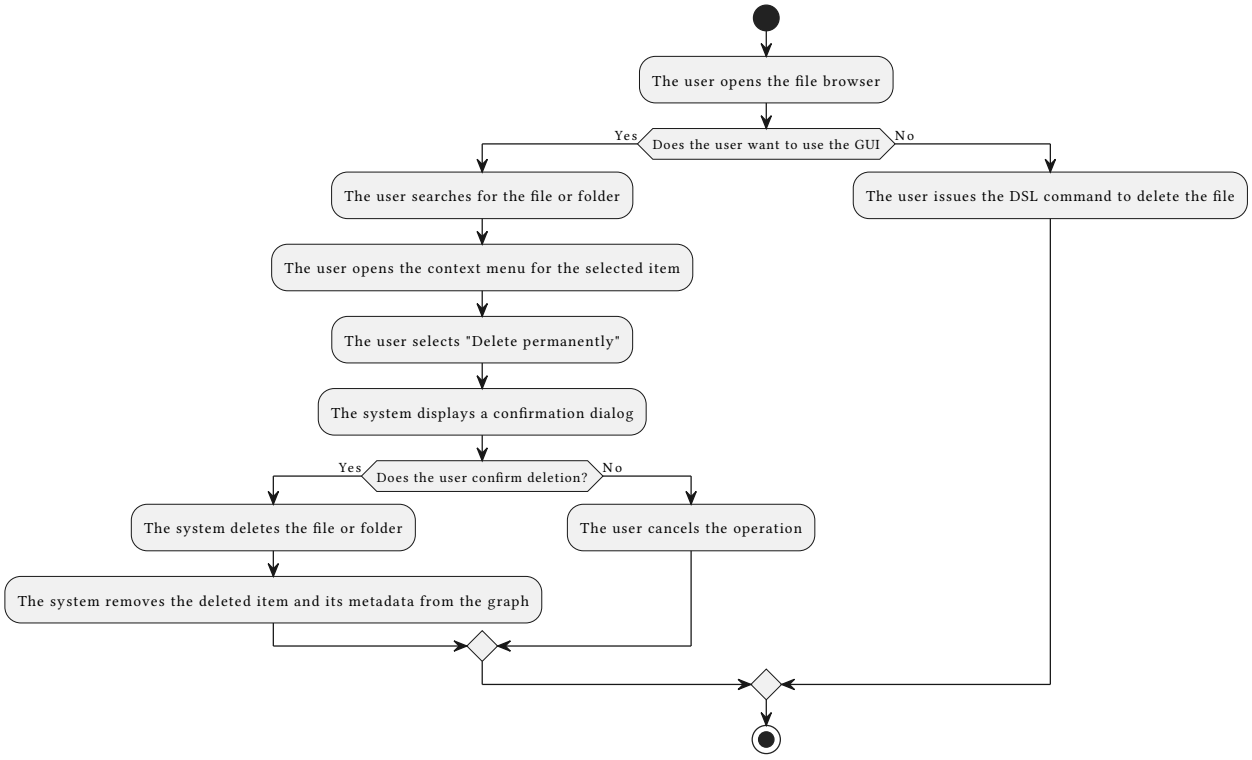


Figure 6: Delete a file activity diagram

2.4.6 Browse a remote filesystem

Preconditions

- The user has the application installed locally
- `GraphleManager` is running on a remote machine
- The user can access the remote machine over SSH
- SSH port forwarding is configured from the local machine to the remote `GraphleManager` port

Flow

1. The user starts the SSH port forwarding connection
2. The user configures the GUI to connect to the locally forwarded server port
3. The user opens the Graphle GUI on the local machine
4. The system connects to the remote `GraphleManager` instance through the forwarded port
5. `GraphleManager` opens the configured home location on the remote machine
6. The system displays files, folders, tags, and relationships from the remote filesystem
7. The user browses the remote filesystem in the same way as in Section 2.4.1
8. When the user opens a file, the GUI downloads a local copy

Alternative flow

- 4a) If the SSH tunnel is not available, the system shows a connection error
- 4b) The user restarts or fixes the SSH port forwarding connection and returns to step 3
- 8a) If the backend is configured as local, the GUI opens the file directly instead of downloading a copy

Postconditions

- The user can browse files from the machine where `GraphleManager` runs
- File paths are interpreted on the backend machine, not on the local GUI machine

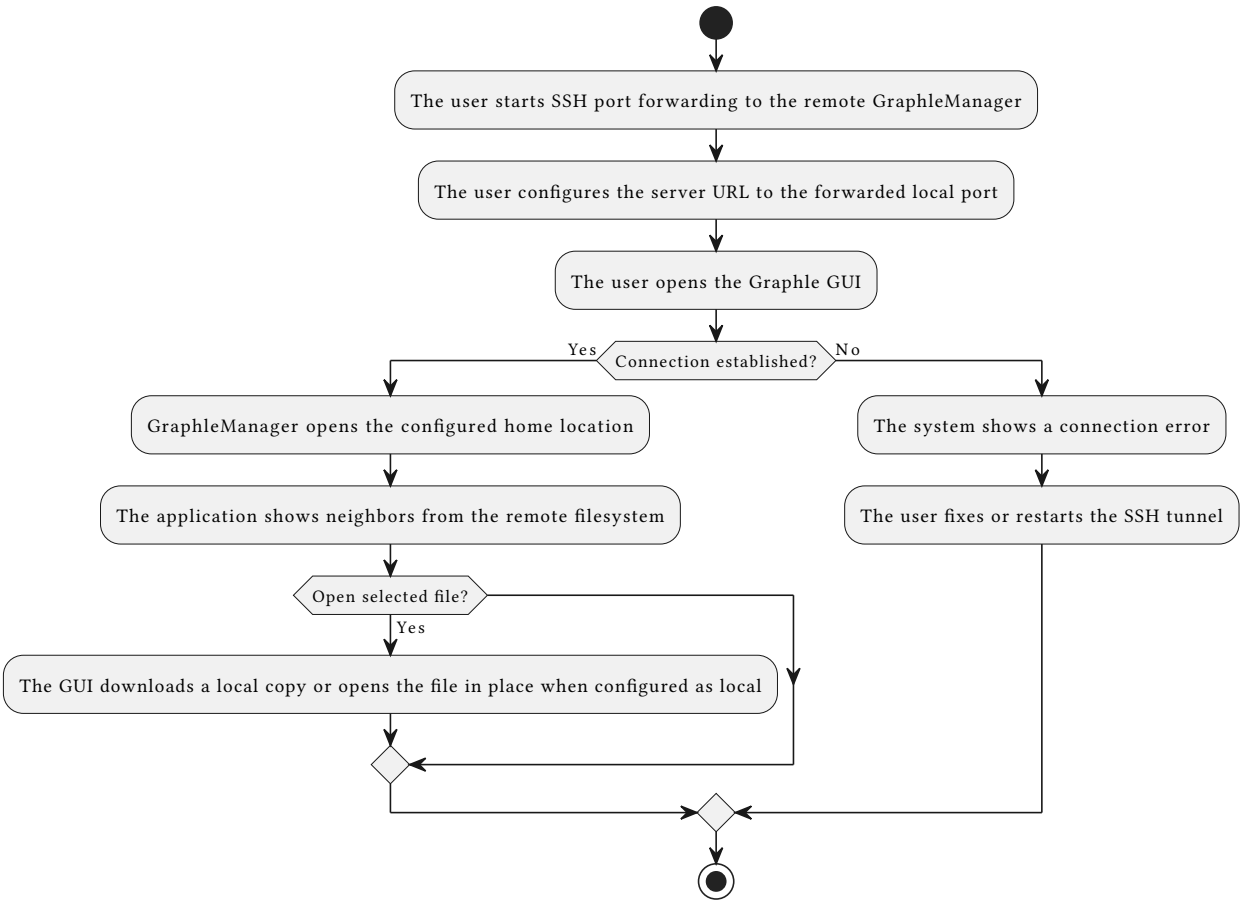


Figure 7: Browse a remote filesystem activity diagram

2.4.7 Add a relationship

Preconditions

- a) The user has the application open
- b) The file browser is connected to a Graphle server
- c) The user wants to connect two files with a relationship

Flow

1. The user searches for a file
2. The user opens the context menu for the selected file
3. The user selects the operation “Add relationship”
4. The system displays a dialog where the user can enter information about the relationship
5. The user enters the information and submits it

Alternative flow

- 1a) The user issues a DSL command
- 5b) The user cancels the operation

Postconditions

- a) The system remembers the new relationship between two entities
- b) The relationship has the name and the value set by the user
- c) The user can traverse the relationship to find the second file directly from the first file

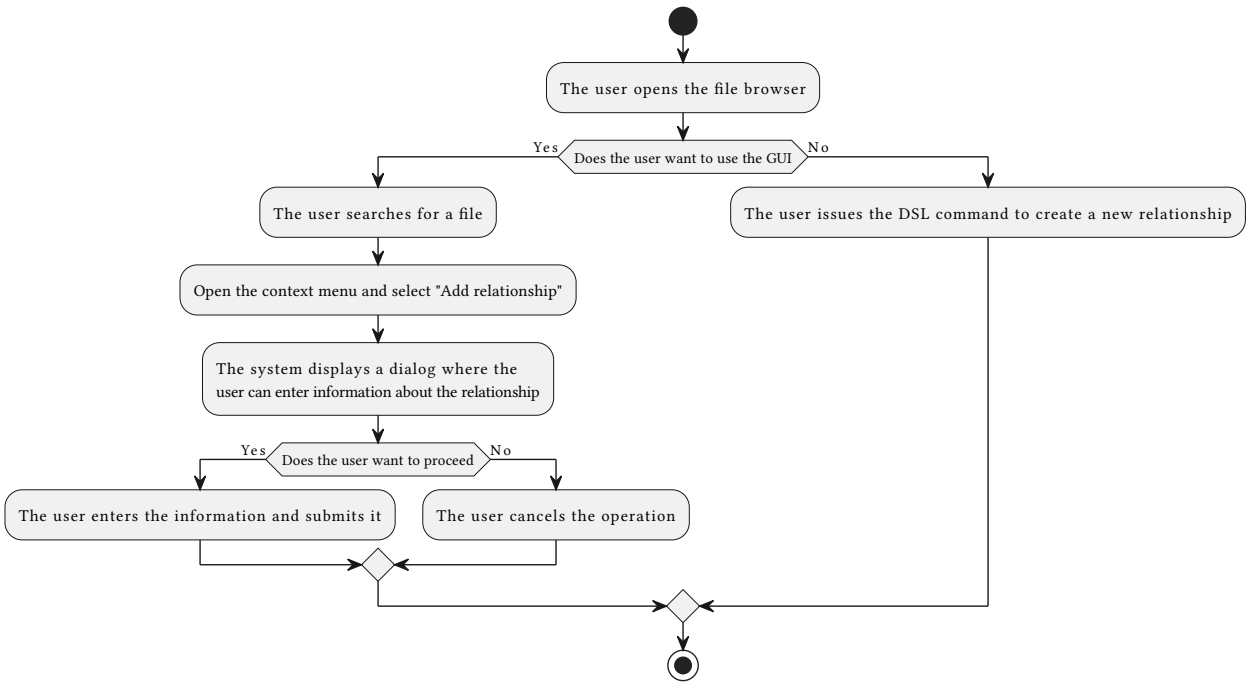


Figure 8: Add a relationship activity diagram

2.4.8 Remove a relationship

Preconditions

- a) The user has the application open
- b) The file browser is connected to a Graphle server

- c) The user wants to remove a relationship between two files

Flow

1. The user searches for a file
2. The user opens the context menu on the desired relationship
3. The user selects the operation “Remove relationship”

Alternative flow

- 1a) The user issues a DSL command
- 2a) The user closes the context menu without selecting an operation

Postconditions

- a) The system removes the relationship between two entities

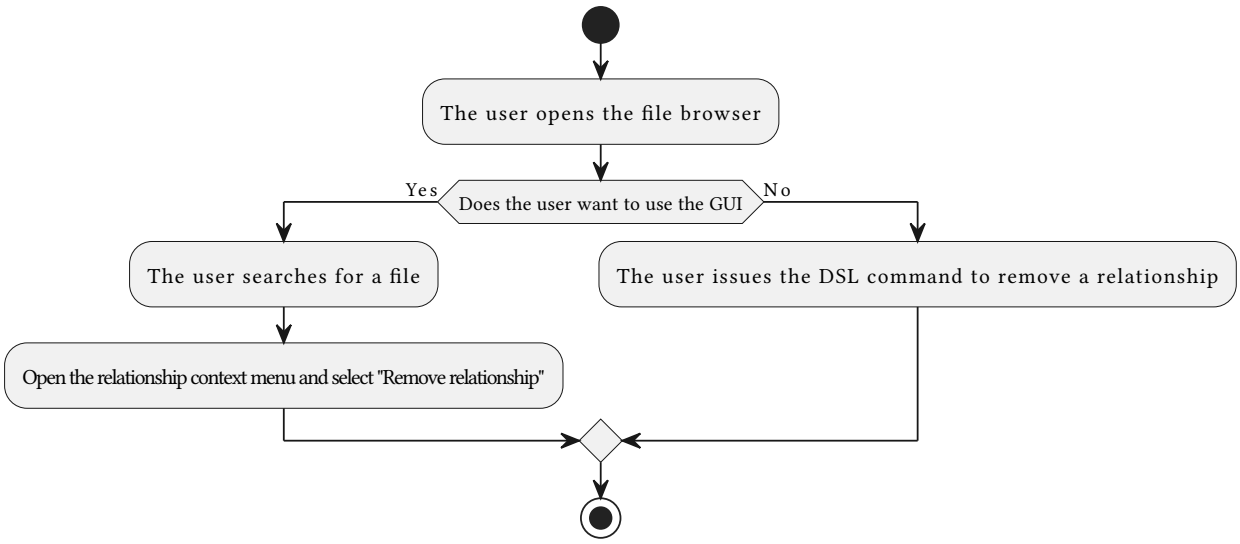


Figure 9: Remove a relationship activity diagram

2.4.9 Add a tag

Preconditions

- a) The user has the application open
- b) The file browser is connected to a Graphle server
- c) The user wants to tag a file

Flow

1. The user searches for a file
2. The user opens the context menu for the selected file
3. The user selects the operation “Add tag”
4. The system displays a dialog where the user can enter information about the tag
5. The user enters the information and submits it

Alternative flow

- 1a) The user issues a DSL command
- 5b) The user cancels the operation

Postconditions

- a) The system remembers a new tag
- b) The tag has a name set by the user
- c) The user can find the file by looking at files tagged with such a tag

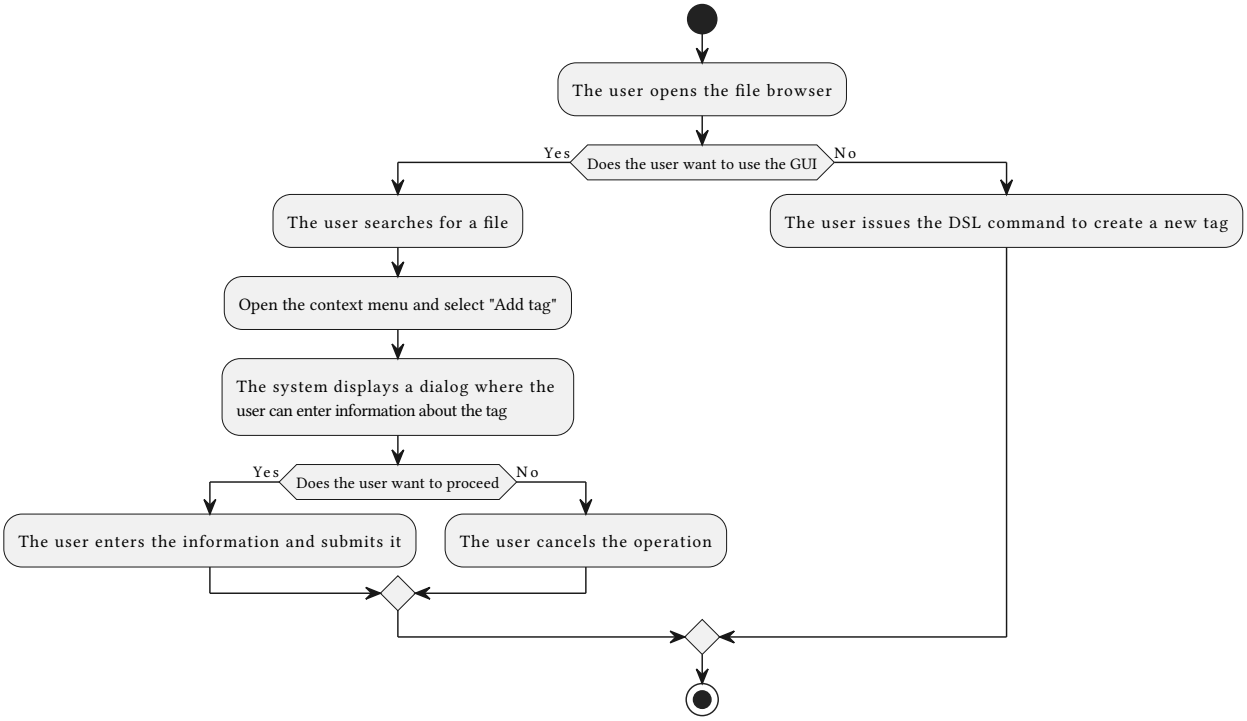


Figure 10: Add a tag activity diagram

2.4.10 Remove a tag

Preconditions

- a) The user has the application open
- b) The file browser is connected to a Graphle server
- c) The user wants to remove a tag from a file

Flow

1. The user searches for a file
2. The user opens the context menu on the desired tag
3. The user selects the operation “Delete”

Alternative flow

- 1a) The user issues a DSL command
- 2a) The user closes the context menu without selecting an operation

Postconditions

- a) The system removes the tag

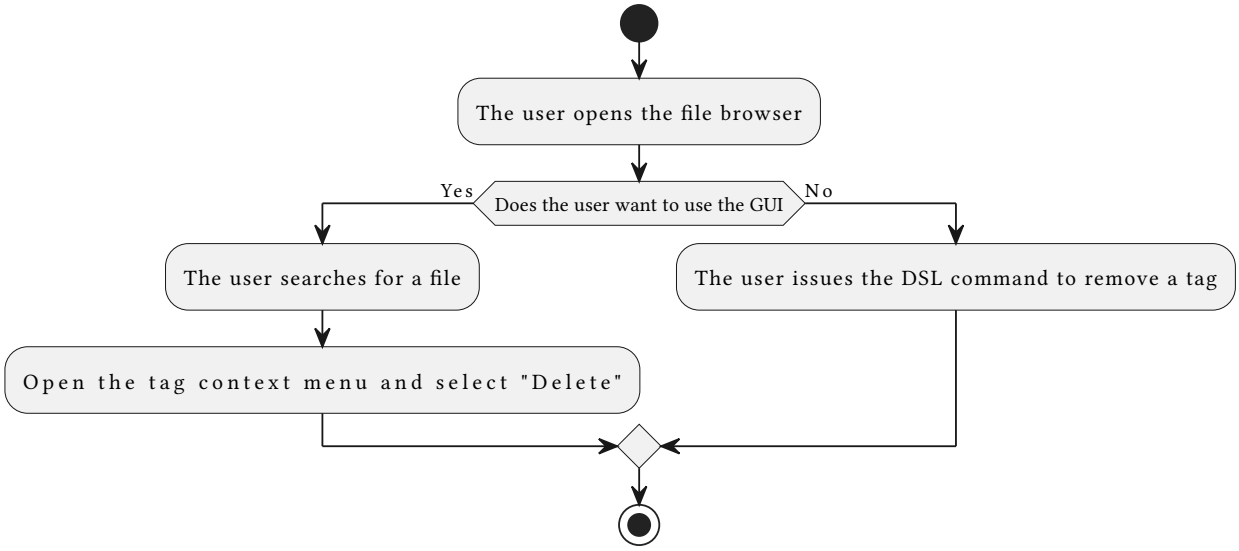


Figure 11: Remove a tag activity diagram

2.4.11 Complex DSL query construction

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The user wants to perform a complex search using multiple criteria

Flow

1. The user opens the DSL command interface
2. The user starts constructing a query with filters they want to apply (more on that in the user tutorial)
3. The user submits the command
4. The system validates the query syntax
5. The system executes the DSL query
6. The system returns the result of the command

Alternative flow

- 4a) If the query syntax is invalid, the system shows an error message and the user returns to step 2

Postconditions

- The user receives a list of objects matching all specified criteria

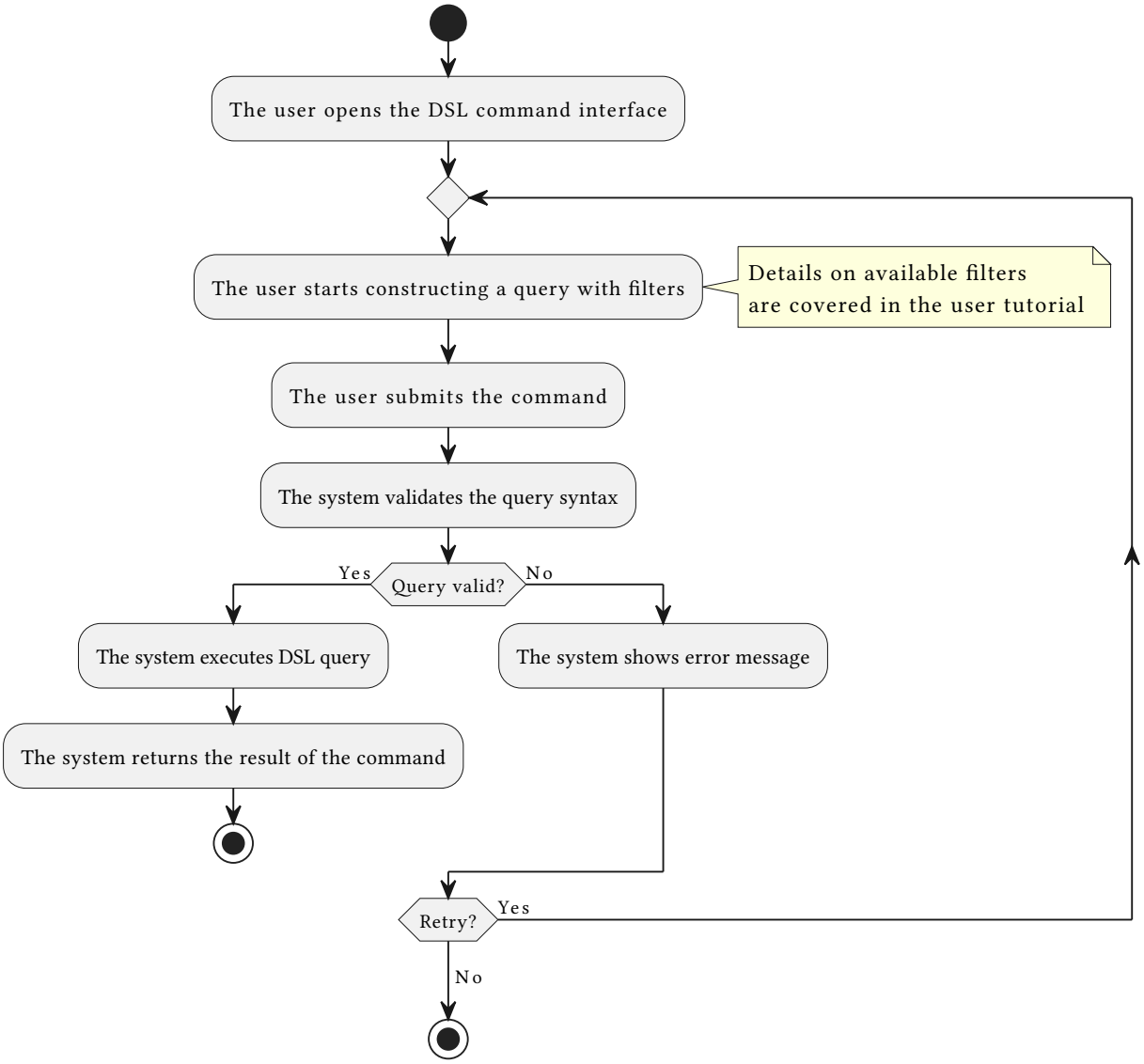


Figure 12: Complex DSL query construction activity diagram

2.4.12 Command auto-completion

Preconditions

- The user has the application open
- The file browser is connected to a Graphle server
- The DSL command interface is active

Flow

1. The user opens the DSL command interface
2. The user starts typing a command
3. The system analyzes the current input prefix
4. The system queries the auto-completion service
5. The system generates suggestions
6. The system displays a list of suggestions to the user

7. The user selects a suggestion from the list or continues typing
8. If the user selects a suggestion, the system completes the command with the selected text
9. The user continues constructing the command or executes it

Postconditions

- The user has a valid or partially constructed DSL command
- The user saves time by not typing complete paths or commands manually

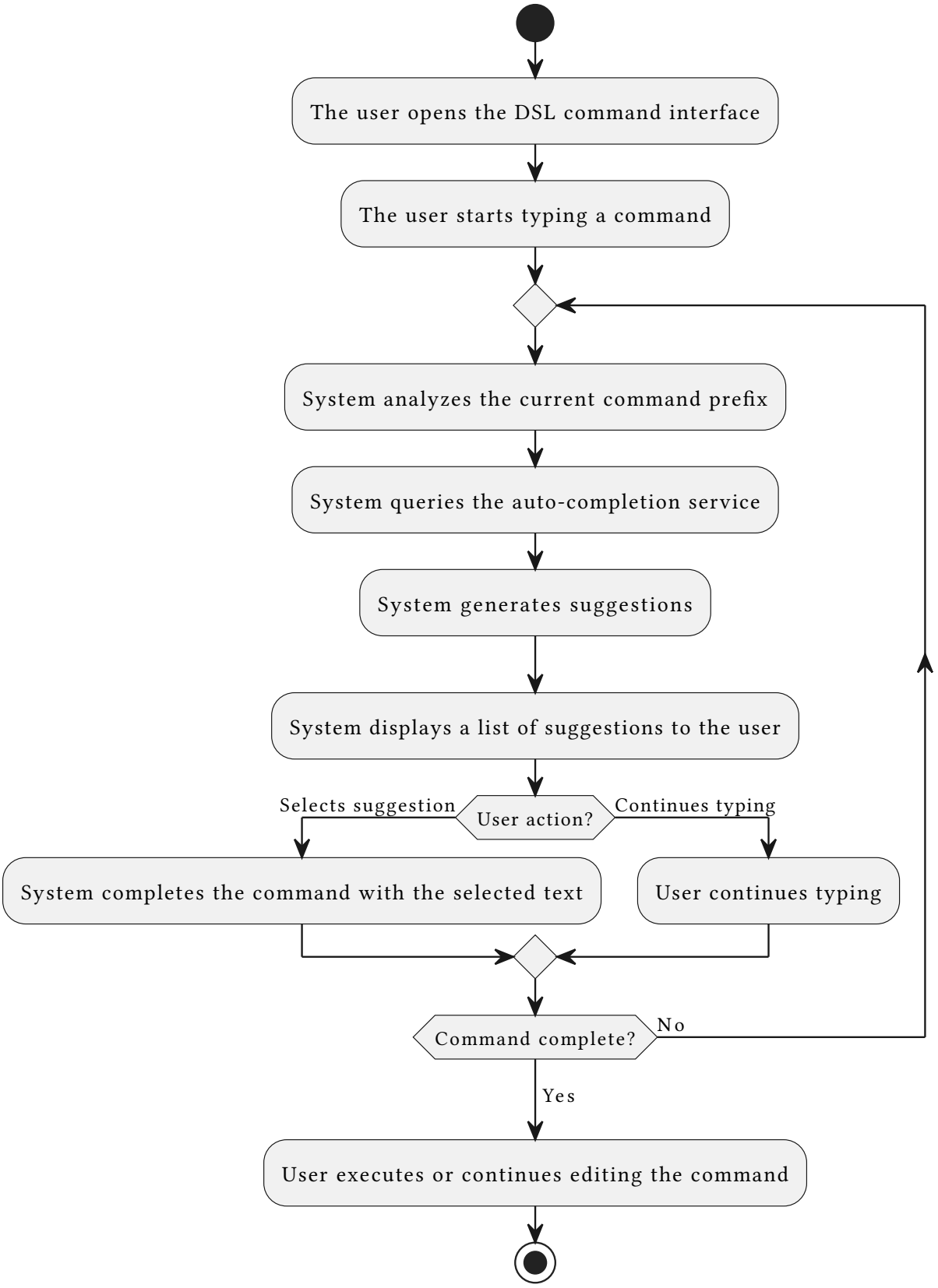


Figure 13: Command auto-completion activity diagram

Section 3

Current landscape

This section examines existing tools that partially address the same user needs as Graphle. The comparison is structured around both the functional requirements established in the Functional Requirements section and the qualitative constraints established in Qualitative Requirements. The criteria therefore cover two groups. The first group consists of support for typed relationships, tags, queries or a query DSL, autocomplete, file preview, and whether the tool can operate on arbitrary files and folders rather than only on a special document type. The second group consists of cross-platform availability, remote use, multiuser support, and storage model. These criteria are not meant to cover every feature of the surveyed tools. They are selected because they correspond to the requirements that most directly determine whether an existing tool can replace Graphle.

3.1 Traditional File Managers

To understand the problem space, we need to assess the current solutions of file managers. Finder for macOS [3], Nautilus [4], and Dolphin [5] for Linux desktop environments were chosen as representatives of file managers. All three provide a graphical directory browser with file manipulation support. They operate on arbitrary files and folders and inherit the permission model of the operating system account used to run them. Finder exposes the macOS tagging system, which persists tags as extended filesystem attributes. Nautilus and Dolphin offer no comparable built-in tagging. All three support remote filesystem access. Finder connects via *Connect to Server* (supporting SFTP (SSH File Transfer Protocol) and SMB (Server Message Block)), while Nautilus and Dolphin accept `sftp://` URIs directly, allowing users to browse files on remote machines and NAS devices as if they were local. They also provide previews or quick viewing workflows for common file types, which makes them stronger than Graphle for direct file inspection.

The closest approximation to relationships available in all three managers is the symbolic link (a filesystem entry that points to another file or directory path), which can make a file appear at a different location. Symbolic links require no special application support. However, they are untyped, as they carry no label describing why the two locations are related, and the filesystem keeps no index of what points to a given target, making them non-queryable as a first-class concept. Finding all files symbolically linked to a given target requires a full recursive filesystem scan, which does not scale.

Dolphin exposes an integrated Konsole panel [6] that can be toggled inside the file manager window, giving users a terminal session whose working directory follows the GUI's current folder. The standard shell commands available, such as `cd`, `mv`, and `find`, can be used to navigate the filesystem. However, they only operate on paths and cannot navigate based on semantic information. Therefore, Dolphin receives a partial rating for the DSL part.

3.2 Obsidian

Obsidian [7] is a personal knowledge management application built on top of a local folder of Markdown files. Its central feature is a graph view that visualizes the `wikilink` connections users embed in their notes, establishing explicit, browsable relationships between documents. Because links are stored as plain text, the vault remains readable without the application. Tags are supported via `#hashtag` syntax or YAML front matter. Its primary storage model is a local Markdown vault plus indexes built by the application and its plugins.

Despite the graph view, Obsidian is scoped to Markdown files. It cannot express relationships between arbitrary file types such as PDFs, images, or source code files, and folders cannot be tagged or linked as first-class entities. Because Obsidian stores its link index only for Markdown vaults, non-Markdown files in the same folder are invisible to the graph, hence the *Partial* rating for arbitrary files and folders in the functional comparison table below. The Dataview [8] community plugin adds a query language over file metadata (data that describes another object, such as tags or relationships describing a file), but its queries are still confined to Markdown files. PDFs, images, or binaries cannot appear as a node in a Dataview query.

3.3 TagSpaces

TagSpaces [9] is a cross-platform file manager that adds a tagging layer directly on top of the existing filesystem. Rather than maintaining a separate database, it encodes tags either in the file name itself (e.g. `report[tax 2024].pdf`) or in a sidecar JSON (JavaScript Object Notation) file placed next to the target, keeping the metadata portable and readable without the application. It supports any file type and includes a built-in viewer for common formats such as images, PDFs, and plain text. This storage model is optimized for tags rather than graph traversal.

TagSpaces has no concept of directed, typed connections between individual files. Files that share a tag can be gathered through a tag-intersection search, but there is no way to express that *document A cites document B* or that *image X was exported from project Y*. Its query model is limited to tag and filename matching, and there is no composable DSL for queries spanning multiple edges. The application has no remote-access capability.

3.4 TMSU

TMSU [10] is a CLI tagging tool that stores tags in its own database and exposes them through a virtual filesystem. Files remain unchanged in their original locations, while still allowing users to persist and query tags. TMSU supports simple tags, tag values, and its own DSL, making it a relevant example of using tags over an ordinary filesystem.

The limitation is that the model remains centered on tags. It cannot express typed relationships such as *derived from*, *cites*, or *belongs to project* between two files or folders. TMSU also has no dedicated graphical interface, no application-level remote-access workflow, and no shared multiuser model. It therefore demonstrates the usefulness of tag-based navigation, but does not cover the graph-oriented file management model required by Graphle.

3.5 autojump

autojump [11] is a shell utility that learns which directories a user visits frequently and allows rapid navigation to them by typing only a partial directory name. Internally it maintains a weighted database of visited paths. The `j <query>` command jumps to the highest-ranked directory whose name contains the query string. This directly addresses quick navigation without requiring the user to remember or type full paths.

autojump is limited to directories and operates only on what the shell has explicitly visited. It carries no concept of relationships between files or directories, no tagging, and no means of querying the filesystem beyond frequency-ranked prefix matching. It is a navigation aid rather than a file management tool, and it provides no graphical interface. It is useful as evidence that users benefit from cross-hierarchy navigation, but its storage model is based on an access frequency rather than a semantic model of the filesystem.

3.6 Comparison

Tool	**Relationships**	**Tags**	**Arbitrary files/ folders**	**File preview**	**Query/ DSL**	**Autocomplete**	**Remote Access**
Finder	No	Yes	Yes	Yes	No	No	Yes
Nautilus	No	No	Yes	Partial	No	No	Yes
Dolphin	No	No	Yes	Partial	Partial	No	Yes
Obsidian	Partial	Yes	Partial	Partial	Plugin	No	No
TagSpaces	No	Yes	Yes	Yes	No	No	No
TMSU	No	Yes	Yes	No	Yes	No	No
autojump	No	No	Folders only	No	No	Yes	No
Graphle	**Yes**	**Yes**	**Yes**	**No**	**Yes**	**Yes**	**Yes**

Table 1: Functional comparison of existing tools against selected requirements.

Tool	Platform fit	Storage model	Multiuser support
Finder	macOS only	Native filesystem plus macOS metadata such as tags and symbolic links	Uses macOS accounts and file permissions, no separate application users
Nautilus	Linux / GNOME	Native filesystem plus GVfs-backed remote locations	Uses Linux accounts and file permissions, no separate application users
Dolphin	Linux / KDE	Native filesystem plus KIO-backed remote locations	Uses Linux accounts and file permissions, no separate application users
Obsidian	macOS, Linux and Windows	Markdown vault plus application and plugin indexes	Primarily single-user local vaults, shared editing depends on optional sync or external storage
TagSpaces	macOS, Linux and Windows	Filenames or sidecar JSON metadata next to files	Local or shared-folder use, access control depends on the underlying filesystem
TMSU	macOS and Linux	Local tag database	Primarily per-user local use, sharing depends on the underlying files and database coordination
autojump	macOS and Linux	Local weighted database of visited directories	Per-shell-user history database, no shared multiuser model
Graphle	**macOS and Linux**	**Existing filesystem plus Neo4j graph metadata and Valkey autocomplete index**	**OS accounts and SSH can separate users. No application-level multiuser authorization in the thesis version**

Table 2: Operational comparison of existing tools against selected requirements and constraints.

None of the surveyed tools satisfies the full requirement set. Standard file managers provide good browsing and file-operation workflows, but their semantic model is limited. Finder exposes OS-level tags, while Nautilus and Dolphin do not provide an equivalent built-in tagging model, and none of the three supports typed, queryable relationships between files as a first-class concept. They are also tied to a particular desktop environment or operating system, so they do not provide one cross-platform graph layer over both macOS and Linux filesystems.

Obsidian is the closest conceptual match, a graph of linked documents with tags and partial query support. However, it is constrained to Markdown notes and cannot work with arbitrary file types in its graph. TagSpaces adds tagging to any file type but has no relationship model and no query DSL beyond tag and filename matching. Its filename and sidecar storage is transparent, but it does not provide a graph storage model. TMSU adds a stronger tag-query and virtual-filesystem model, but remains tag-centric and lacks typed relationships, a GUI, and a remote workflow. autojump demonstrates that traversing links across the whole filesystem is useful for navigation, but it has no file management capabilities, no relationship model, and no GUI.

Graphle addresses the gap left by the surveyed tools by extending a standard filesystem with relationships, tags, a DSL with filename autocompletion, and remote access. It keeps file contents in place, stores semantic graph metadata in a graph database, and supports both files and folders as graph nodes. However, Graphle still lacks features that could be added in future extensions. It does not provide application-level multiuser support, so supporting multiple concurrent users would require running separate GraphleManager instances. Second, it does not yet provide built-in file preview, so users inspect file contents through external applications rather than inside the Graphle client.

Section 4

Design Documentation

This section describes how the functional and qualitative requirements defined in the analysis are realized in the design of the system. It covers the data model, the overall architecture and communication between components, the technologies selected for each layer, and the visual design of the GUI client. Each design decision is traced back to the requirements from the analysis that it addresses.

4.1 Data Model

The application works with a graph-oriented extension of the user's existing filesystem. The filesystem itself remains outside the application data model. It stores file contents, permissions, timestamps, and the directory hierarchy. Graphle stores only the metadata needed for graph navigation and derives the rest from the live filesystem when a query is executed.

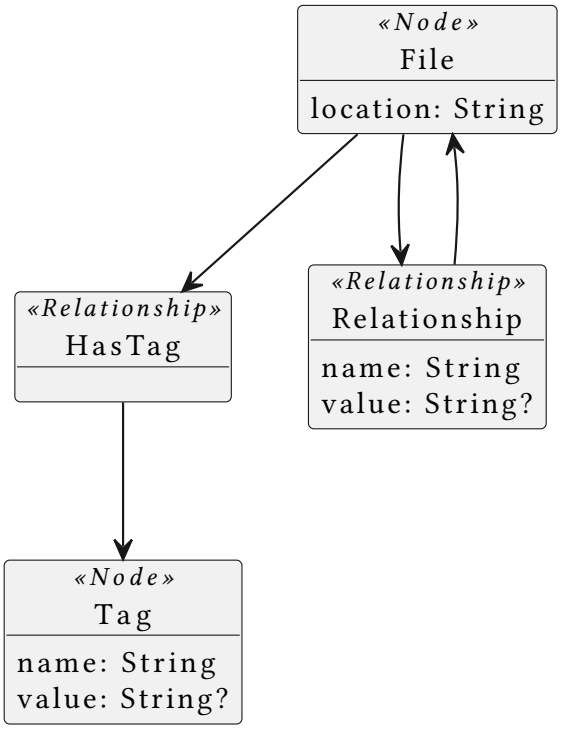


Figure 14: Graphle graph data model

The Figure 14 contains two node types and two edge types. `File` is the central node type. Its `location` attribute is the absolute path on the machine where `GraphleManager` runs and acts as the logical identifier used by the API, DSL, and UI. The database does not duplicate the file's binary content or operating-system metadata. This keeps the model compatible with existing filesystem tools and supports lazy loading (loading data only when it is requested instead of eagerly loading everything in advance) (F1, F9).

`Tag` represents a reusable categorization label. It has a required `name` and an optional `value`. The value allows both simple tags, such as `project`, and key-value tags, such as `priority = high`. A file is connected to a tag by a `HasTag` edge. Tags that are no longer connected to any file can be removed without changing the filesystem.

`Relationship` represents a semantic relationship between two files or folders. It is stored as a directed edge from one `File` node to another and contains a required `name` and an optional `value`. The direction is part of the model, so a relationship from `A` to `B` does not imply the reverse relationship. When a user requests a bidirectional relationship, the application represents it as two directed `Relationship` edges with the same attributes.

The filesystem hierarchy is intentionally not persisted as graph metadata. Parent and descendant relations are computed from the current filesystem state and returned together with stored semantic relationships. This means that moving or deleting a file outside Graphle does not immediately corrupt a stored hierarchy. The next query or background sweep can reconcile the graph with the actual filesystem.

The public API exposes `File` as an aggregate view containing its `location`, `tags`, and visible `connections`. Additional response types, such as `FilesByTagResponse` and DSL responses, are transport shapes rather than independent domain entities. They exist so clients can display query results and mutation outcomes without introducing new persisted concepts into the data model.

4.2 Architecture

The system is composed of five components: GraphleManager, GraphleUI, the DSL client, Neo4j, and Valkey. The application does not use a single communication style. GraphleManager exposes three distinct communication interfaces:

- GraphQL (a query language for APIs that allows clients to request exactly the data they need) handles structured file operations.
- REST (Representational State Transfer) over HTTP (Hypertext Transfer Protocol) handles DSL execution and file downloads.
- WebSocket carries autocomplete traffic.

The following sections describe how each component and interface addresses the functional and qualitative requirements defined in the analysis.

4.2.1 Components

GraphleManager is the backend service and the single point of contact for all clients. It hosts the DSL interpreter, manages both external datastores, and exposes the three communication interfaces described below. Because GraphleManager is a network service, clients can connect to an instance running on a remote machine, giving users access to a remote filesystem (Q1.3). It is implemented on the JVM, which together with Compose Multiplatform on the client side ensures the application runs on both macOS and Linux without duplicating the logic (F10). When an auxiliary component such as Valkey or a connected GUI becomes unavailable, GraphleManager continues to handle all other operations. Failures in optional subsystems are caught and logged without propagating to the core (Q3.2).

GraphleUI is the desktop GUI client built with Compose Multiplatform (F6). It communicates with GraphleManager exclusively through the public API, which means it can be replaced by any third-party client that speaks the same interfaces (Q1.2). The default client ships with both a light and a dark theme (Q1.2). File and folder manipulation is exposed through GraphQL mutations for the GUI and through the DSL for the command line client (F4, F5).

DSL client is a command line tool that sends commands to GraphleManager over REST (F7). It is the second user interface available to users alongside the GUI (Q1.1).

Neo4j stores the graph of files, relationships, and tags as an LPG (F2, F3). Only GraphleManager reads from and writes to Neo4j directly, and clients access the database layer through it. However, the database instance is exposed on a configurable port with a published schema. This allows external applications and independent forks of GraphleManager to query or even extend the dataset directly, fulfilling the extensibility requirement (Q4.1).

Valkey holds the filename autocomplete index in memory as a trie (a tree data structure where each node represents a character, used for efficient prefix-based lookups) (F8). GraphleManager populates and queries this index on behalf of clients. Frequently accessed trie nodes are additionally cached inside GraphleManager using a concurrent cache (a faster temporary storage layer used to avoid repeatedly reading the same data from a slower source) layer, so hot nodes are served without a network round trip to Valkey. If Valkey is unavailable, autocomplete is silently disabled while all other operations continue (Q3.2).

4.2.2 Communication Interfaces

GraphQL (`/graphql`) is used by GraphleUI for all file, tag, and connection operations. It allows the client to request exactly the fields it needs, which is important when a query result can contain a large number of files. The schema also gives any future client full transparency over the available data and operations.

REST (`/dsl`, `/download`) is used by the DSL client and by GraphleUI for DSL command execution and file downloads. REST was chosen for these endpoints because issuing an HTTP request from the command line is straightforward, whereas constructing a GraphQL query would add unnecessary overhead for CLI use.

WebSocket (`/ws`) provides a persistent connection for DSL autocomplete. Keeping the connection open avoids establishing a new TCP handshake on each keystroke, which is necessary to meet the latency requirement (Q2.1). GraphleUI reconnects automatically with exponential back-off if the connection is lost (Q3.1).

4.2.3 Background Maintenance and Lazy Loading

Files, parent directories, and descendants are not indexed eagerly. They are loaded from the filesystem on demand the first time a query requests them (F9). This avoids a potentially unbounded startup scan and keeps the graph populated only with content the user has actually navigated to. Parent and descendant relationships are resolved live by reading the OS filesystem hierarchy rather than being stored in Neo4j, meaning any file already

on disk is immediately accessible through Graphle without first being loaded into the database (F1).

A background job, **Neo4JSweeper**, runs inside GraphleManager at a configurable interval. It fetches all file locations recorded in Neo4j, checks whether each path still exists on the filesystem, and removes any that have been deleted outside the application along with their orphaned tags and relationships. This keeps the database consistent with the underlying filesystem without requiring the user to manually clean up deleted files.

4.3 Module Decomposition

This section adds a C4 view of the system. The diagrams define the architectural boundaries that follow from the requirements and complement the component-level design. The module names follow the current Gradle modules and package structure of `GraphleManager` and `GraphleUI`, so the diagrams can be traced back to the implementation.

4.3.1 System Context

At the highest level, Graphle is a desktop-oriented file management system that extends the user's existing filesystem with graph metadata. The operating filesystem remains the source of truth for file existence, hierarchy, and file contents. Graphle stores only semantic metadata in Neo4j and data used for autocomplete in Valkey.

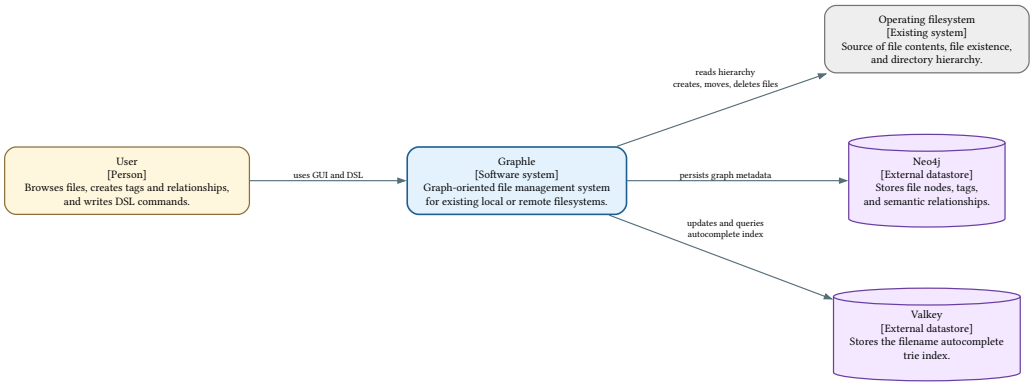


Figure 15: C4 system context diagram

The Figure 15 separates Graphle from the external systems it depends on. This separation is important for backward compatibility (F1), because files can still be manipulated by normal operating-system tools. It is also important for component isolation (Q3.2), because the autocomplete datastore can fail without preventing the core graph and filesystem operations from running.

4.3.2 Containers

The system is split into a desktop GUI container and a backend container. `GraphleUI` is a Kotlin Compose Multiplatform application. It communicates with `GraphleManager` through public network interfaces only: GraphQL for file, tag, and relationship operations, REST for DSL execution and file downloads, and WebSocket for command autocomplete.

`GraphleManager` is a Spring Boot service that coordinates filesystem operations, the Neo4j graph, and the Valkey-backed autocomplete index.

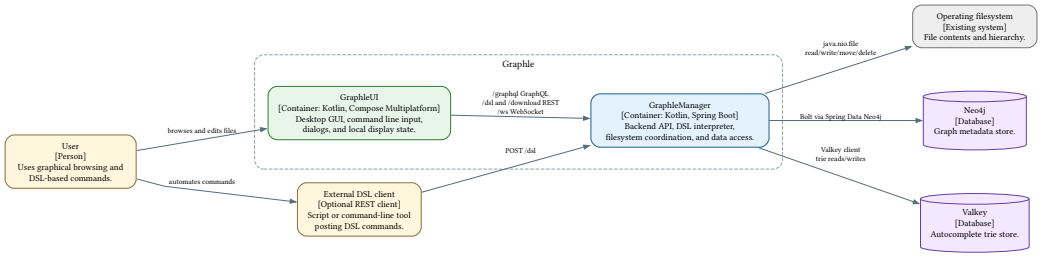


Figure 16: C4 container diagram

The Figure 16 shows that the GUI is not coupled to the database and filesystem directly. This supports remote access (Q1.3) and extensibility (Q4.1), as another client can reuse the same public API instead of having to rely on the current state of backend internals.

4.3.3 GraphleManager Modules

The backend module decomposition follows the Gradle modules in `GraphleManager`. The API adapters accept client traffic and delegate to service modules. The service modules contain the application logic and decide whether an operation needs Neo4j, the live filesystem, Valkey, or a combination of them.

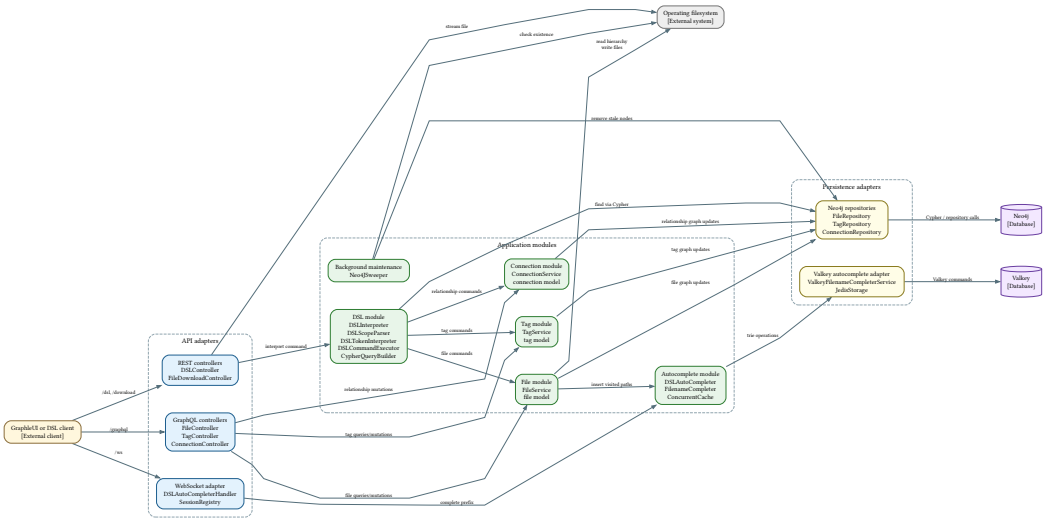


Figure 17: C4 component diagram of GraphleManager

The Figure 17 highlights the main backend boundaries. The file module is the only module that directly writes to the filesystem. The tag and connection modules are graph-specific and persist through Neo4j repositories. The DSL module reuses the same domain services for commands that change state, while complex `find` queries are translated to Cypher (the declarative query language used by Neo4j to express graph patterns) and executed against Neo4j. The autocomplete module owns the Valkey-backed trie and is updated asynchronously from file navigation.

4.3.4 GraphleUI Modules

The frontend module decomposition follows the packages in `GraphleUI/src/main/kotlin/com/graphle`. `App.kt` owns the displayed state and delegates rendering to the header, dialog, and body modules. All backend communication is kept in transport helpers, which lets the composable UI code work with local models instead of raw GraphQL or HTTP payloads.

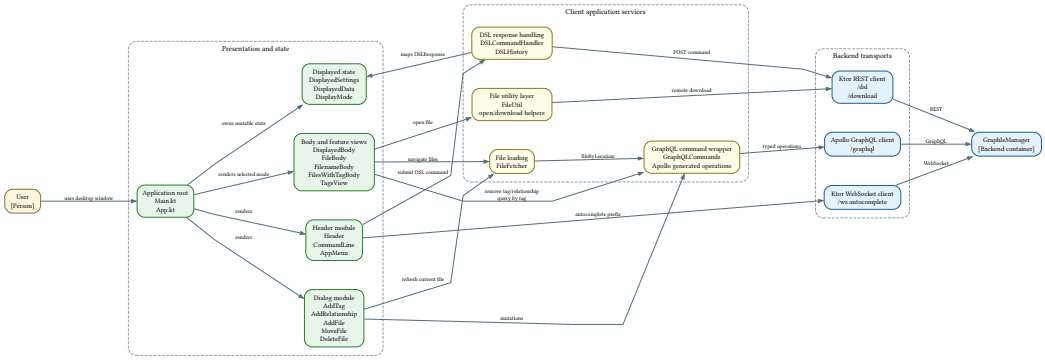


Figure 18: C4 component diagram of GraphleUI

The Figure 18 shows how the GUI keeps its visible content in one shared state model. Actions initiated from the command line, dialogs, or file views are sent to the backend through the appropriate transport. The returned data is converted to `DisplayedSettings`, which `App` then uses to select and update the currently displayed body. As a result, graphical workflows and DSL-driven workflows update the interface through the same state path.

4.4 Used Technologies

The selection of technologies was guided by three criteria that apply to all components: cross-platform compatibility to ensure the application runs on macOS and Linux without platform-specific defects (requirement F10), ecosystem maturity and community size to lower the barrier for future contributors and reduce the risk of relying on abandoned dependencies, and fit for the specific technical constraints of each component, such as latency requirements or data-model alignment. Where multiple candidates satisfied the first two criteria, the decision was made on technical fit.

4.4.1 Programming Language

The project targets macOS and Linux (requirement F10), so the chosen language must be fully cross-platform. This ensures users do not encounter configuration-specific bugs, reducing the risk of failures that would go undetected during routine developer testing on a single platform.

The language must also be suitable for building large-scale desktop GUI applications and have a sufficiently large community. Since other developers should be able to extend the project with new features, a niche language without an established ecosystem is not suitable. At the time of project inception, the only candidates meeting both constraints were Java [12] and Kotlin [13].

Kotlin was chosen over Java because it supports every Java library while also offering additional language features. This gives developers access to a wide range of existing functionality, allowing them to focus on the core domain rather than reimplementing general-purpose utilities. Kotlin's type system is also more expressive than Java's. Illegal states can be made unrepresentable at compile time more easily, which makes client-server communication less error-prone by enforcing a more precise contract.

4.4.2 API

The application is divided into a backend service and two types of clients: a GUI client and a DSL client. Both clients communicate exclusively through the API, which allows either to be swapped out or extended by third-party developers, and enables the backend to serve remote filesystems over a network without architectural changes.

The backend is implemented with Spring Boot [14] and exposes three interfaces. The GUI client communicates via GraphQL [15], which allows it to request exactly the data it needs. This is important since a query result can contain millions of files, and fetching all fields when only a filename is needed would be wasteful. The DSL client communicates via REST, since the DSL is also intended to be used from the command line, where issuing REST calls is simpler than constructing GraphQL queries. Lastly, the backend also provides a WebSocket endpoint used by the GUI to deliver low-latency autocomplete suggestions while the user types DSL commands.

Spring Boot provides a mature ecosystem for building production-grade JVM services, including first-class GraphQL and REST support. Its widespread adoption ensures that contributors with prior JVM experience can onboard with minimal friction.

4.4.3 GUI

The graphical client is built with Compose Multiplatform [16], a major GUI framework for Kotlin. Beyond being a widely adopted option, which eases onboarding for new contributors, Compose Multiplatform supports sharing UI code across desktop, web, and mobile targets from a single codebase. This makes a future port to other platforms a low-cost extension rather than a full rewrite.

4.4.4 DSL

The application provides a DSL for advanced filesystem operations. Because autocomplete for the DSL must react to each keystroke, its parser runs inside the backend service and communicates available completions via WebSockets. WebSockets maintain a persistent connection, avoiding the overhead of establishing a new HTTP connection on every keystroke, which is critical for meeting the low-latency autocomplete requirement. gRPC was considered as an alternative, but its additional infrastructure, protobuf schema definitions, and code generation would be overkill for a single stream exchanging short strings. There are specific tools for creating DSLs. However, as the auto-completer needs to parse the commands either way and needs to fetch data from the database, the parser and completer were kept in the same service to avoid serialization between different services, which is important to keep the response time to a minimum.

4.4.5 Relationships Between Files

Storing and querying arbitrary relationships between files requires a database whose native data model is a graph. Two graph data models were considered: the LPG and the Resource Description Framework (RDF).

RDF was eliminated for two reasons. First, it mandates the use of URIs as node and relationship identifiers. This imposes an unnecessary burden on users who manage a purely local filesystem with no intent of publishing or interlinking it with external datasets. Second, RDF stores are typically triple stores that materialize graph structure only at query time, whereas LPG databases index edges natively, resulting in faster traversal for the relationship-heavy workloads this project targets [17].

After selecting the LPG model, several self-hosted databases capable of storing property graphs were compared: ArangoDB, ArcadeDB, and Neo4j. ArangoDB [18] and ArcadeDB [19] are multi-model databases that combine graph storage with document and other data models, which would be useful if Graphle stored several kinds of data in one database. Graphle only persists semantic graph metadata, so a dedicated graph database that stores the data directly as a graph is a better fit. Furthermore, Neo4j [20] was selected because it is a mature self-hosted LPG database with the broadest adoption [21], Cypher support, and direct Spring integration through Spring Data Neo4j [22].

4.4.6 Autocomplete

As specified in requirement Q2.1, autocomplete responses must be delivered within the threshold of human reaction time, which is approximately 250 ms [23]. Meeting this constraint requires that candidate filenames be held in memory rather than retrieved from disk on each keystroke.

Valkey [24], a Linux Foundation fork of Redis [25], was selected as the in-memory store. Valkey is reported to be faster and more memory-efficient than Redis in benchmarks [26] and has received corporate backing from Oracle, AWS, and Google. Organizations now depend on it in production, which reduces the risk of the project being abandoned [27]. Developers familiar with Redis can apply that knowledge directly to Valkey with no additional learning overhead.

To support infix completion while keeping memory consumption reasonable, filenames are indexed using a modified trie. Leaf-level path components are stored without their parent segments, enabling efficient prefix search across all components of a filename. Parent path information is preserved as a back-reference attached to leaf nodes, so a full path can be reconstructed from any matching suffix without duplicating the entire path at every entry.

4.5 Mockups

This section presents the visual design of the GUI client. Each mockup illustrates a key screen of the application and explains the layout.

4.5.1 File Detail Page mockup

The main page opens at the user's home directory and displays the file detail view. The header contains a hamburger menu and a command line input pre-filled with the current

DSL command. Below the header, the body is divided into three sections: URLs (tags whose value is a URL), Tags, and Files. Files are rendered as pills. Parent and descendant connections use arrow icons to indicate direction, while named relationships show their label. This page is displayed as the detail view for every file, which can be traversed by pressing the pill or by using the DSL.

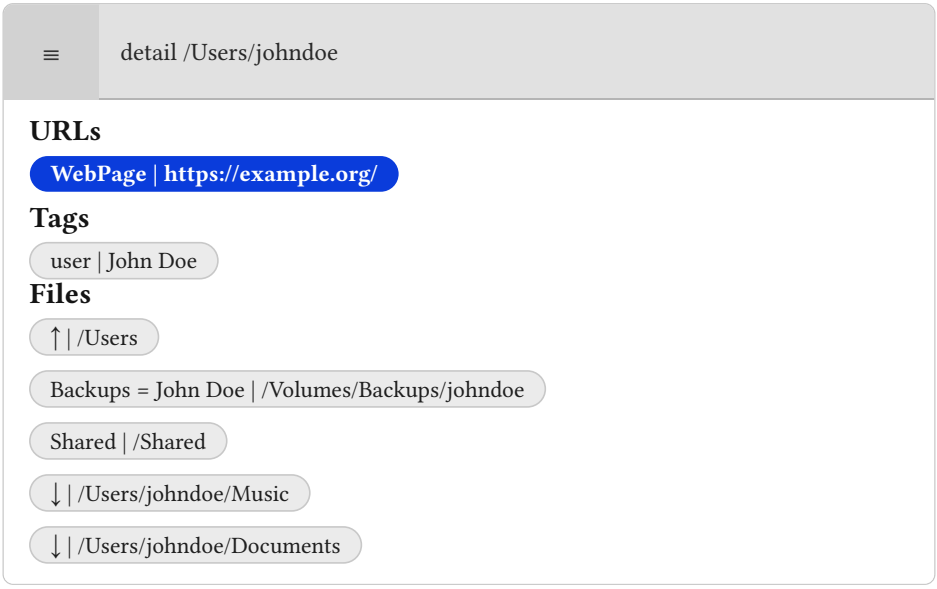


Figure 19: Main page - file detail view

4.5.2 Filenames Results mockup

The filenames results page displays the output of a find DSL command ending with filenames scope. The header shows the executed query, and the body lists the matching files as pills.

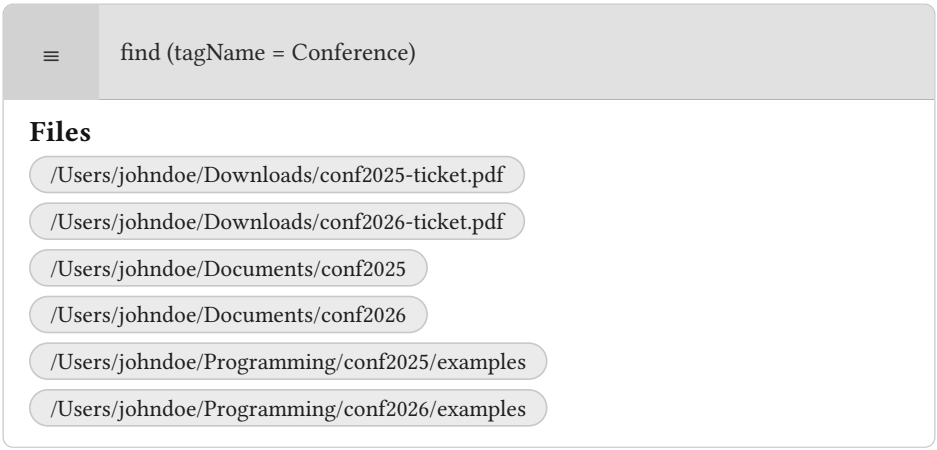


Figure 20: Main page - filenames view

4.5.3 Find with Relationship Filter mockup

When a find command includes a relationship filter, the results page shows matching files as pills. Each pill displays the relationship name and value alongside the file path.

Figure 21: Main page - find results with relationship filter view

4.5.4 Files With Tag mockup

The files with tag page displays the output of a tag lookup command. Each result pill shows the matched tag name and value followed by the file path.

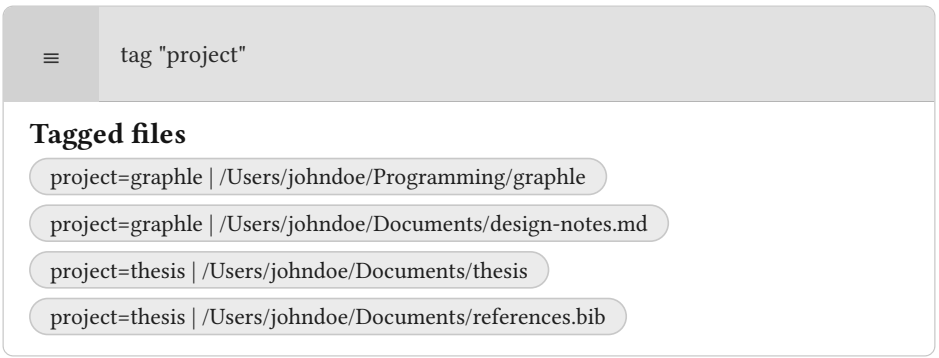


Figure 22: Main page - files with tag view

Section 5

Developer Documentation

This chapter describes the implementation of Graphle for a developer who wants to work with the source code. It complements the design chapter, which explains why the system is structured the way it is, by focusing on how that structure is implemented. The codebase consists of two applications: `GraphleManager`, a backend written in Kotlin and Spring Boot, and `GraphleUI`, a desktop client written in Compose Multiplatform. These applications can be developed independently, and the only contract the developers need to adhere to is the API communication defined in a subsequent section.

The chapter begins with the package layout of both applications and the concrete classes, repositories, and services that make up each layer. It is followed by a sequence-diagram walkthrough of the most representative end-to-end flows. It then outlines all API endpoints in detail, providing paths, methods, parameters, and example payloads across the GraphQL, REST, and WebSocket interfaces. Next, it describes the algorithms behind the core features, the modified trie used for autocomplete, the background sweeper that keeps the graph consistent with the filesystem, and the multi-stage DSL interpreter pipeline. It also discusses the problems that arose during implementation. Finally, the chapter concludes with the testing strategy and the continuous-integration setup that ensures the codebase keeps working as future contributors extend it.

The implementation follows the design chapter without major deviation, and the requirements defined in the analysis are fulfilled by the resulting program, as summarised in Table 3.

Req.	Design component	Implementation	Test
F1	File module	`FileController`, `FileService`, `FileRepository`	`FileIntegrationTest`
F2	Connection module	`ConnectionController`, `ConnectionService`, `ConnectionRepository`	`ConnectionIntegrationTests`
F3	Tag module	`TagController`, `TagService`, `TagRepository`	`TagIntegrationTests`
F4	File module	`FileController`, `FileService`, `FileRepository`	`FileIntegrationTest`
F5	File module	`FileService`, `FileController`	`FileIntegrationTest`
F6	GraphleUI	`GraphleUI` project	Manual
F7	DSL module	`DSLController`, `DSLInterpreter`, `CypherQueryBuilder`, `DSLCommandExecutor`	`DSLInterpreterTest`, `DSLInterpreterIntegrationTest`
F8	Autocomplete module	`FilenameCompleter`, `DSLAutoCompleter`, `DSLWebSocketManager`	`ValkeyFilenameCompleterTest`
F9	File module	`FileService`, `Neo4JSweeper`	`Neo4JSweeperTest`
F10	Chosen stack, File module	`FileService`	Manual
Q1.1	GraphleUI containers + DSL module	`FileController`, `TagController`, `ConnectionController`, `DSLController`	`FileIntegrationTest`, `TagIntegrationTests`, `ConnectionIntegrationTests`, `DSLInterpreterIntegrationTest`
Q1.2	Public API, GraphleUI theme	`GraphQLCommands`, GraphleUI `common/`	Manual
Q1.3	Boundary between GraphleManager and GraphleUI, download endpoint	`FileDownloadController`, `DSLRestManager`	`FileDownloadControllerTest`
Q2.1	Autocomplete module	`FilenameCompleter`, `DSLWebSocketManager`	Latency measurement
Q3.1	WebSocket transport	`DSLWebSocketManager`	Manual
Q3.2	Module boundaries (Figure 17)	`FileService`, `TagService`, `ConnectionService`	`FileIntegrationTest`, `TagIntegrationTests`, `ConnectionIntegrationTests`
Q4.1	DSL module, parser/dispatch split	`DSLInterpreter`, DSL parser	`DSLInterpreterTest`

Table 3: Requirements traceability: each requirement mapped to its design component, key implementation class or module, and automated test.

5.1 Backend

The backend service is `GraphleManager`, a JVM Spring Boot application written in Kotlin.

5.1.1 Module and Package Layout

`GraphleManager` is a Gradle multi-project build. The root project contains the Spring Boot entry point `GraphleManagerApplication.kt`, startup configuration, the background sweeper, and resources. Backend code is split into subprojects so cross-domain dependencies are visible in `build.gradle.kts`. `GraphleManager` consists of:

- `common`: shared utilities.
- `model`: serializable API and Neo4j data classes shared by the domain modules.
- `tag`: tag persistence, tag service operations, and the tag GraphQL controller.

- `connection`: relationship persistence, connection service operations and the connection GraphQL controller.
- `autocomplete`: Valkey-backed filename trie storage, caching, and the autocomplete service contract.
- `file`: filesystem interactions, file-node persistence, file mutations and queries, and downloads.
- `application`: cross-domain orchestration.
- `dsl`: DSL parsing and execution, the `/dsl` REST endpoint, and the autocomplete WebSocket.

5.1.2 Domain Layer

`File`, `Tag`, and Connection are declared as Spring Data Neo4j entities with UUID identifiers generated by the driver. The `File` node stores the file's absolute path and is linked to its tags and connections. The `Tag` node has an indexed name and an optional value so the same name can take different values on different files. A Connection is a named, directional edge between two `File` nodes with an optional value and a bidirectional flag. Hierarchical parent and descendant relationships are not persisted and are instead derived from the live filesystem at query time.

Index creation is handled once on startup by `init/Neo4JConfig`, which ensures the mentioned indexes exist.

5.1.3 Repository Layer

Each entity has a `Neo4jRepository` declaring the Cypher queries the rest of the code needs:

- `FileRepository`: `addFileNode`, `removeFileByLocation`, `moveFile`, `getFileLocationsByConnections`.
- `TagRepository`: `tagsByFileLocation`, `addTagToFile`, `removeTag`, `filesByTag`, `removeOrphanTags`.
- `ConnectionRepository`: `neighborsByFileLocation`, `addConnection`, `removeConnection`.

5.1.4 Service Layer

Services sit between the controllers and the repositories and carry all non-trivial logic. `FileService` performs filesystem I/O through `java.nio.file.Files`, reconciles database state with disk state after `addFile`/`removeFile`/`moveFile`, and schedules asynchronous autocomplete trie updates via `insertFilesToCompleter`. It exposes functions `fileType`, `descendantsOfFile`, and `parentOfFile` that let the controllers compute hierarchical neighbors without going through the database (F1, F9). `TagService` and `ConnectionService` are thin wrappers that delegate to their repositories. Background work is dispatched on a different thread so the caller does not need to wait and its failure will not affect the rest of the system.

5.1.5 Controller Layer

The GraphQL surface is split across three `@Controller` classes, one per entity: `FileController`, `TagController`, and `ConnectionController`. Their methods are tagged

with the standard query and mutation annotations, and the nested `File.tags` and `File.connections` fields are fetched lazily, only when the client actually requests them. The `fileByLocation` query is implemented outside those domain controllers in `application/FileDetailsController`, which delegates to `FileDetailsService`. This service assembles a single `File` response from the live hierarchical neighbors, the persisted custom connections, and the file's tags. The separation is enforced by Gradle module dependencies, meaning only the application module can access other domain modules. Exceptions thrown inside any of these controllers are caught by `@GraphQlExceptionHandler` methods that translate `IOException` and `FileAlreadyExistsException` into readable GraphQL errors used by the UI.

Three more endpoints sit outside GraphQL. `DSLController` is a `@RestController` mapped at `/dsl` that hands the request body to the DSL interpreter and returns the resulting response. `FileDownloadController` serves raw file contents over `GET /download?path=…` with a detected MIME type. The GUI uses it whenever the backend is running on a different host. A WebSocket endpoint at `/ws` carries autocomplete traffic.

5.1.6 Configuration

The backend reads its settings from `application.properties`, which exposes:

- `server.port`: the HTTP port on which the service listens (default 5824). The GUI's `server.port` must match this value for the transports to reach the backend.
- `spring.graphql.graphiql.enabled`: toggles the built-in GraphiQL playground on the same HTTP port.
- The Neo4j connection: bolt URL and credentials consumed by Spring Data Neo4j.
- The Valkey connection: host and port that the autocomplete layer dials for trie storage.
- `neo4j.sweeper.interval`: how often the background sweeper reconciles the graph with the live filesystem (default one minute).
- `cache.ttl`: the expiration applied to Valkey keys held by the autocomplete trie (default thirty days).

5.2 Frontend

The frontend is `GraphleUI`, a desktop application built with Compose Multiplatform in Kotlin. It targets the JVM and is packaged for supported platforms. Backend access is split between two transports. Apollo (a GraphQL client library used by GraphleUI to generate and execute typed GraphQL operations) handles all file, tag, and connection operations. Ktor handles the DSL REST call and the autocomplete WebSocket.

5.2.1 Package Layout

All source lives under `com.graphle` and is split by feature. `Main.kt` opens a single Compose `Window` around the root `App` composable, which delegates body rendering to a small dispatcher keyed on the current display mode. The feature packages are:

- `common/`: configuration, singletons, the GraphQL wrapper, a trash helper, the UI state models, and the shared theme and reusable visual primitives.

- `file/`, `tag/`, `fileWithTag/`: one package per main entity, each holding its data model alongside the composables that render it. The file package also collects the fetching, opening, and downloading helpers it needs.
- `dialogs/`: the modal flows for create/delete/move/tag/relationship actions and for surfacing error and invalid-file messages, plus a composite that mounts them inside `App`.
- `header/`: the search header (command line, theme toggle, menu) and the REST and WebSocket transports it drives.
- `dsl/`: command history and the response dispatcher that turns a DSL reply into a display mode change.

5.2.2 Configuration

The GUI reads a YAML configuration file with a single `server` section that exposes two options:

- `port`: the TCP port on which the backend is listening. Used to build the GraphQL, REST, download, and WebSocket URLs that the transports target.
- `localhost`: a boolean indicating whether the backend runs on the same host as the GUI. When true, the file opening flow skips the HTTP download step and opens the file in place. When false, the GUI always pulls a local copy through `/download` first.

5.2.3 State Management

The UI is driven by a single source of truth held in `App.kt`: a reactive variable `displayedSettings` holding `DisplayedSettings`. `DisplayedSettings` wraps a nullable `DisplayedData` (the currently visible entities: filenames, tags, connections, or a file detail) and a `DisplayMode` enum that selects which body to render. `DisplayedBody.kt` branches on the mode and delegates to the feature-specific page type. Child composables never hold the state themselves. `App` hands them paired getter and setter lambdas (`getDisplayedSettings`/`setDisplayedSettings`, and analogous ones for the theme), so the source of truth stays unique and any update, whether from a dialog, a menu entry, or a body composable, flows back through the same write path.

5.2.4 Backend Communication

Apollo generates a typed Kotlin client from the server's `schema.graphqls`. `common/GraphQLCommands.kt` exposes generated operations and maps the Apollo DTOs onto the UI-local models.

`DSLRestManager` posts DSL commands to `POST /dsl` on the configured backend and deserialises the `DSLResponse`. `DSLCommandHandler` dispatches on `DSLResponse.type`, chooses the matching `DisplayMode`, and calls an `onEvaluation` callback to update the UI state. `DSLHistory` keeps the last successfully displayed command so the UI can re-submit it after a transient failure (Q3.1).

Real-time autocomplete uses its own connection. `DSLWebSocketManager` keeps a single session against `/ws`, exposes a flow of incoming suggestions, sends outgoing messages, and reconnects with exponential back-off. This keeps the socket hot across keystrokes so latency is dominated by server-side lookup rather than TCP handshake (Q2.1).

5.3 Sequence Diagrams

This section walks through four representative end-to-end flows. The first two are the most common write and read paths a user issues from the GUI and the DSL client. The third describes how the autocomplete loop meets the 250 ms latency budget (Q2.1). The fourth shows the background work that keeps the database consistent with the filesystem (F9). In every diagram, solid arrows represent synchronous calls and dashed arrows represent returned values. The ↺ glyph marks work performed on the actor's own lifeline (an asynchronous task, a disk write, or an internal computation).

5.3.1 Adding a File with a Tag

The "add file" dialog is triggered from the GUI and a tag is attached to the newly created file. The first round trip creates the file on disk and the `File` node in Neo4j, and, as a side effect, asks the autocomplete service to index the new path. The second round trip merges the tag node and the `HasTag` edge. After both complete, the GUI re-fetches the file detail to display the updated tag set.

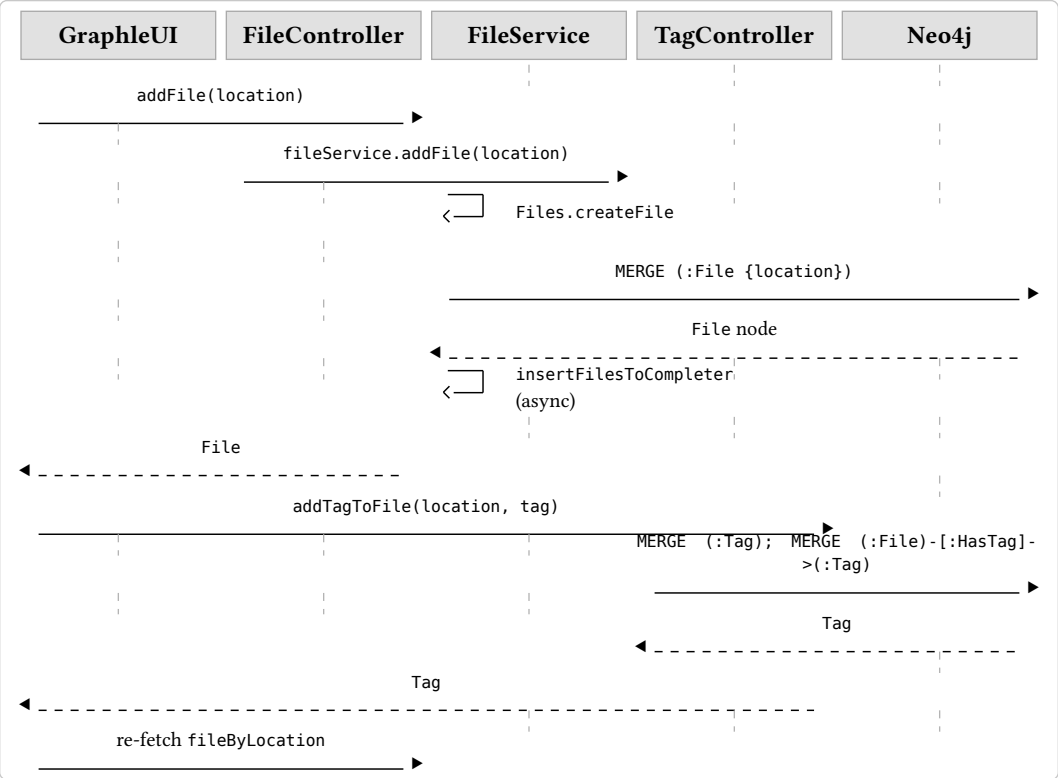


Figure 23: Adding a file and attaching a tag.

5.3.2 DSL `find` Query

The DSL client posts a command string to `/dsl`. `DSLInterpreter` selects the `find` branch, and the query is split into file- and relationship-level scopes by `DSLScopeParser`, compiled into Cypher by `CypherQueryBuilder`, and executed by `DSLCommandExecutor` against Neo4j. The response type (`FILENAMES` vs `CONNECTIONS`) is derived from the last scope of the query so that the client can choose a matching `DisplayMode` without a second round trip.

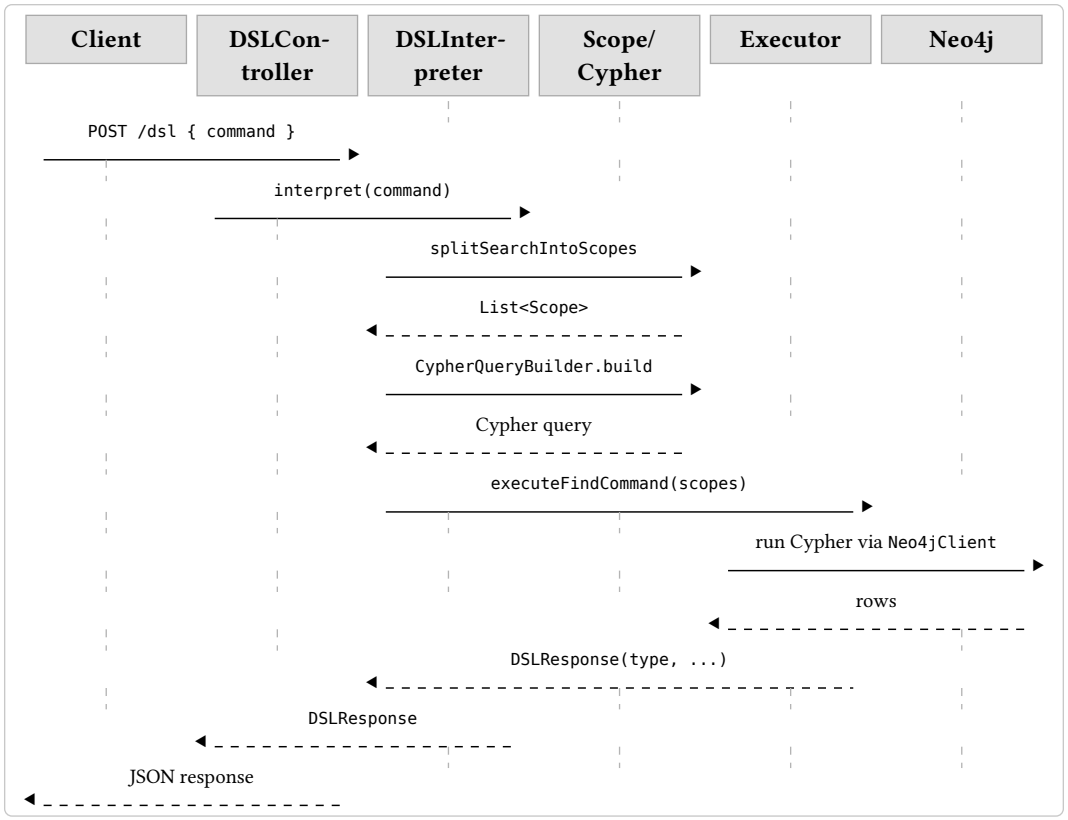


Figure 24: Executing a DSL `find` query.

5.3.3 Autocomplete Keystroke

Every keystroke in the DSL command line sends the current prefix over a persistent WebSocket. `DSLAutoCompleter` first classifies the command kind from the opening token, then asks `FilenameCompleter` for up to five completions. The filename completer reads its trie primarily from an in-process `ConcurrentCache` and falls back to Valkey only on a miss. Every Valkey access also refreshes the key's TTL so hot entries stay warm for `cache.ttl`.

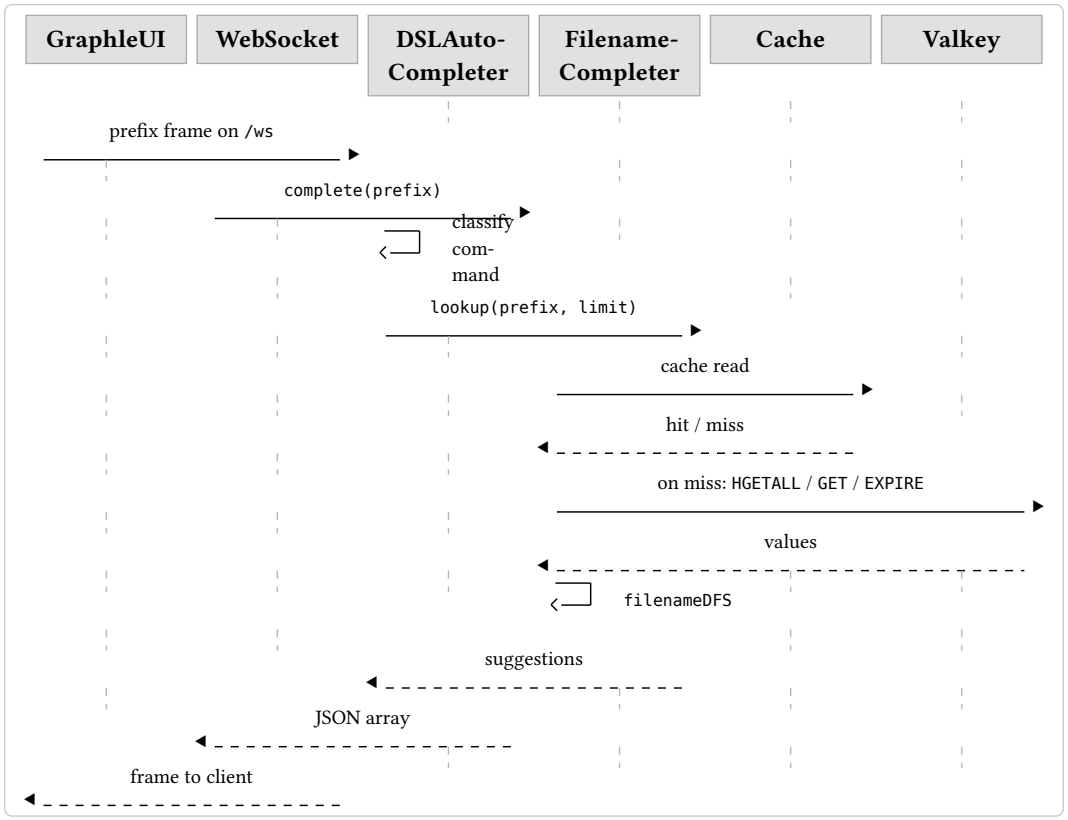


Figure 25: Autocomplete suggestion on a keystroke.

5.3.4 Background Sweeper

`Neo4JSweeper` runs in its own coroutine and wakes up every `neo4j.sweeper.interval`. It lists all `File` nodes, tests each for filesystem existence through an injectable predicate, removes the stale ones with `DETACH DELETE`, and then prunes any tag nodes that lost their last incident edge. The sweeper is the only component that reconciles the database with external filesystem changes, which is what allows the rest of the system to treat Neo4j as eventually consistent with the disk.

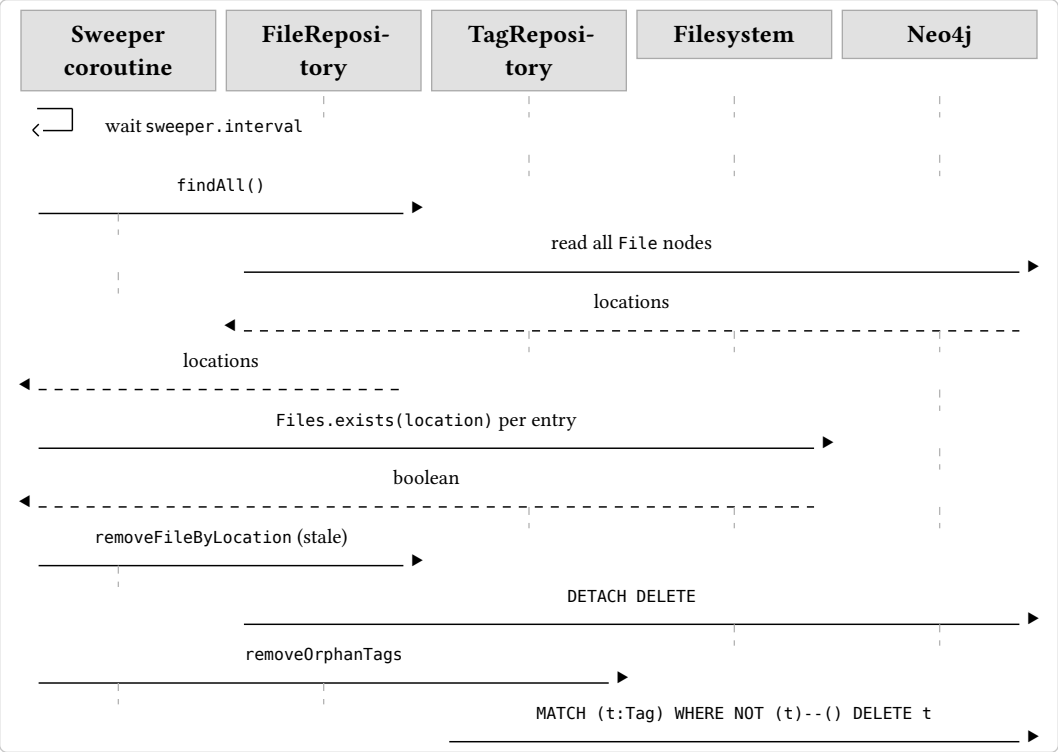


Figure 26: Background sweep of orphaned database state.

5.4 API Documentation

`GraphleManager` exposes three interfaces on the same HTTP port: GraphQL at `/graphql` for structured CRUD operations, REST under `/dsl` and `/download` for the command-line DSL client and for file transfers, and a WebSocket at `/ws` for real-time autocomplete. All three share the same Spring process and the same security model.

5.4.1 GraphQL

The schema is declared in `GraphleManager/src/main/resources/graphql/schema.graphqls` and is the authoritative source for every type and field. GraphiQL is enabled in `application.properties`, so the schema can be explored interactively at `http://<host>:5824/graphiql`.

Query	Purpose
`fileByLocation(location, showHiddenFiles)`	File detail with tags and all live neighbors (parent, descendants, custom connections).
`fileType(location)`	Returns `File` or `Directory`, or `null` if the path is not present.
`tagsByFileLocation(location)`	Lists every tag attached to the given file.
`filesByTag(tagName)`	Reverse lookup from tag to the files that carry it.
`filesFromFileByRelationship(fromLocation, relationshipName)`	Files reachable from `fromLocation` through a named relationship (custom or hierarchical).

Query	Purpose
`optionsByDslPrefix(dslPrefix, limit)`	Autocomplete suggestions for a DSL prefix, offered over GraphQL as an alternative to the WebSocket.
`entriesFromDSL(dslCommand, limit)`	Executes a DSL command from GraphQL and returns an `Entries { entryTypeName, identifiers[] }` tuple.

Mutation	Purpose
`addFile(location)`	Creates the file on disk, adds the `File` node, schedules an insert into the autocomplete trie.
`removeFile(location)`	Removes the file from disk and deletes its node along with all its edges from the graph.
`moveFile(locationFrom, locationTo)`	Moves the file on disk and updates the stored path.
`addTagToFile(location, tag)`	Creates or updates the tag node and its link to the file.
`removeTag(location, tag)`	Removes the tag link and garbage-collects the tag node if it becomes orphaned.
`addConnection(connection)`	Creates a named edge between two `File` nodes.
`removeConnection(connection)`	Removes a named edge between two `File` nodes.

The `File` type embeds `tags: [Tag!]!` and `connections: [Connection!]!` as nested fields, so a client can request the complete detail in a single round trip. Inputs `TagInput` and `ConnectionInput` mirror their output types. `ConnectionInput.bidirectional` is non-nullable on input so callers must decide whether the new edge should be traversed both ways.

5.4.2 REST

`POST /dsl` accepts `{ "command": "<dsl command>" }` and returns `{ "type": ResponseType, "responseObject": [String] }`. `ResponseType` is one of `FILENAMES`, `CONNECTIONS`, `FILE`, `TAG`, `SUCCESS`, or `ERROR`. The payload shape depends on the type:

- `FILENAMES`: absolute paths.
- `CONNECTIONS`: JSON-serialized `Connection` records.
- `FILE` / `TAG`: JSON-serialized `File` / `TagForFile` records.
- `SUCCESS`: empty list, used by mutating commands such as `addFile`, `addRel`, `addTag`.
- `ERROR`: a single human-readable message.

`GET /download?path=<absolute path>` streams the file contents with a detected `Content-Type`, served by `FileDownloadController`.

5.4.3 WebSocket

`/ws` carries the autocomplete protocol used by the GUI. The client opens a single long-lived session, which can reconnect with exponential back-off if it drops, and sends the current DSL prefix on every keystroke. The server answers with a JSON array of suggestions produced by `DSLAutoCompleter`. Keeping the connection persistent avoids a TCP handshake per keystroke, which is what makes the 250 ms budget in Q2.1 achievable.

5.4.4 Authentication

No authentication is performed on any of the three interfaces. GraphleManager is intended to run on the operator's own machine or on a remote host reached through SSH port forwarding, as described by the remote-access requirement (Q1.3). In that model, the forwarded port must remain under the operator's control, so the current version must not be exposed directly to untrusted callers.

Application-level authentication is therefore future work. A complete remote-access security model would need a uniform layer across GraphQL, REST, and the WebSocket handshake, plus authorization checks that map authenticated users to permitted filesystem paths. This planned extension is deliberately out of scope for the current bachelor's thesis implementation.

5.5 Algorithms employed

5.5.1 Autocomplete

Autocomplete is implemented as a trie storing paths. The trie is stored in Valkey, with an in-process cache in front of it to save on round trips to the database. The implementation lives in `dsl/FilenameCompleter.kt` and utilizes a `Storage` abstraction so the trie logic can be tested against a fake or swapped for a different storage backend.

Each trie node is identified by a numeric index, where the root is assigned index 0. Each node stores its parent link back up the trie, the character it represents, a set of back-references to full-path nodes (used for the reverse-lookup described below), and a flag indicating whether it terminates a complete path. A separate top-level counter holds the index of the most recently allocated node so inserts pick a fresh index even across process restarts.

The insertion path implements the "modified trie" described in the architecture chapter. Because many files share long common ancestor paths, storing each full path as an independent sequence of characters would duplicate most nodes. The modified trie avoids this by inserting each path twice. Once as the complete absolute path, and once as only its leaf component, with a back-reference pointing from the leaf to the corresponding full-path node. The effect is that a directory prefix shared by many files is stored once and reused rather than repeated for every full path. The parent chain for any leaf can always be reconstructed by following the back-reference to the full-path node and then walking up to the root. Memory therefore grows with the number of unique directory and filename components in the dataset, rather than with the number of files multiplied by their average depth. This matters because the in-process cache stores trie nodes directly. Duplicated prefixes would fill the cache with redundant entries and reduce its effectiveness.

To validate this behavior empirically, a small experiment was run on files accessible from a home directory. For each sample size, five random subsets were selected and inserted into the trie. The experiment recorded the number of created trie nodes and the sum of the sampled path string lengths before prefix sharing.

Sample size	Trie nodes	Sum of path lengths	Nodes / length
10	555	810	68.5%
100	4,060	7,966	51.0%
1,000	32,911	79,477	41.4%
10,000	272,931	796,071	34.3%
50,000	1,193,662	3,979,503	30.0%

Table 4: Trie size measured on random subsets of files accessible from a home directory.

The results show the expected sharing effect. As the sample grows, more files reuse already indexed directory prefixes, so the number of trie nodes per input character decreases from 68.5% for 10 files to 30.0% for 50,000 files. The autocomplete index still grows with the number of known paths, but the experiment supports that storing shared prefixes once avoids a large amount of redundant path data in realistic directory trees.

Lookup descends the trie character by character from the root, returning an empty result as soon as a character has no child. From the node reached, `filenameDFS` finds full paths by interleaving three sources: the prefix node itself if it is marked `:full`, the `:parents` set, which contains predecessor path indices, and a recursive descent through the children. Every returned path is checked for existence on disk, so completions pointing at files deleted since the last sweeper run are filtered out before the suggestions are returned.

Five dedicated caches, one per access pattern, sit in front of the storage layer. Each cache entry is refreshed on every access (whether a hit or a miss), so hot nodes stay warm without a separate TTL refresh step. The result is that the first read after startup and all subsequent reads produce the same in-memory view of the data.

To prevent node identifiers from being reused, the database maintains a monotonic `last` counter and assigns each newly created trie node the next available key. This gives every node a stable identity for its entire lifetime, including across process restarts, which in turn makes the application's in-process cache safer as a cached key can be trusted to refer to the same logical node rather than accidentally pointing to newly inserted trie state after reuse.

5.5.2 Garbage Collector

The sweeper lives in `sweeper/Neo4JSweeper.kt` and keeps the graph consistent with the underlying filesystem (F9). It runs as a single background task that wakes at a configurable interval and executes `sweep`.

A sweep pass proceeds in two phases. The first phase removes file entries whose recorded locations no longer exist on disk. The second phase removes tag entries that became orphaned as a result of those file deletions.

5.6 DSL

The DSL interpreter is organized as a staged pipeline centered around `DSLInterpreter.interpret`. At the outermost level, the first token identifies the command being invoked. Commands such as file, tag, and relationship updates are parsed into typed inputs and then dispatched to the corresponding application service. The `find` command is structurally richer: rather than mapping directly to a single service operation, it is first parsed, then translated into Cypher, and only afterwards executed against the graph. Keeping this query processing pipeline separate from the dispatch path used by simpler commands makes the language easier to extend. Adding a new command such as a file or tag operation usually requires only a new typed input and a new service branch, while changes to the `find` language remain confined to the parser, query builder, and executor. This reduces coupling between unrelated parts of the DSL and lowers the risk that extending one command family will unintentionally affect another.

Conceptually, a `find` query is an ordered sequence of scopes. Parenthesized scopes describe constraints on file nodes, while bracketed scopes describe constraints on relationships, and the two kinds of scope may be interleaved arbitrarily. This allows a query to express graph traversals incrementally, alternating between conditions on files and conditions on the relationships that connect them. The parser's task is to reconstruct this scoped structure from the input, after which each recovered scope is interpreted as a boolean combination of simple predicates.

Once the query has been decomposed into scopes, `CypherQueryBuilder` translates the DSL representation into Cypher. At this stage, user-facing names and operators are mapped onto the corresponding graph properties and comparison operators, and small mismatches between user syntax and storage format are normalized. For example, numeric comparisons over tag values are coerced into numeric form even though the underlying values are stored as strings. The result is a query that preserves the user's intent while remaining compatible with the graph schema.

Execution is handled by `DSLCommandExecutor`, which submits the generated Cypher to Neo4j and converts the returned rows into a `DSLResponse`. The shape of the response is determined by the final scope of the query. If the query ends in a relationship scope, the caller receives relationships. If it ends in a file scope, the caller receives filenames. This lets a single surface syntax support several related query patterns without requiring separate command families for each return type.

Autocomplete follows the same structural view of the language. Rather than treating the input buffer as an unstructured string, `DSLAutoCompleter` reconstructs the parse state of the partially written command and infers what kind of token is valid at the cursor position. Filename completion is therefore invoked only where a filename is syntactically expected.

5.7 Discussion of implementation problems

5.7.1 Real-time autocomplete

Requirement Q2.1 caps an autocomplete response at 250 ms, which rules out doing per-keystroke network work that scales with the input. A fresh TCP handshake on every keystroke is avoided by keeping a single WebSocket open on `/ws`. The client transparently reconnects with exponential back-off when the link drops, so an established connection is already in place whenever typing occurs. A Valkey round trip per trie node traversed would in turn make lookup cost linear in the prefix length, which is avoided by fronting each access pattern with an in-process cache and by the modified trie encoding described in the Algorithms section.

The implementation is designed to make the 250 ms target realistic for established interactive sessions with warm caches. The manual measurements in the Testing section show that the sampled requests stayed below this threshold. The first few keystrokes after a cold start may exceed the budget while the cache is repopulated from Valkey, and slower responses are also possible under unusual system load or slow filesystem/database access.

5.7.2 Transparent computing

Transparent computing in Graphle means a GUI on one machine can drive a filesystem that lives on another (Q1.3). The invariant that holds this together is that all filesystem I/O is performed on the server. The GUI never assumes a path received from the backend is a local path. Paths are meaningful only on the host where `GraphleManager` runs.

Opening a remote file therefore takes a detour. The GUI calls `GET /download?path=...` against `FileDownloadController`, writes the response into a `GraphleDownloads/` directory under the user's home, and hands that local copy to the operating system. When the backend happens to be running on the same host, a `server.localhost` flag in the YAML config skips the download step and opens the file in place.

5.7.3 Portability

The application targets macOS and Linux (F10). Windows is deliberately not supported. See the analysis chapter for the reasoning behind not supporting it. Because both the Spring Boot backend and the Compose Desktop frontend are written in Kotlin and compile to JVM bytecode, JDK 21 is the single runtime that makes both components portable. JDK 21 is supported on both macOS and Linux. Everything else (file existence, directory enumeration, path normalization) delegates to `java.nio.file`, which abstracts the remaining OS differences.

5.7.4 User navigation

The file-detail view is intentionally the central point of the GUI. `FileDetailsService.fileByLocation` in the `application` module assembles it in a single round trip by pulling from three independent sources: hierarchical neighbors (`descendantsOfFile`, `parentOfFile`) come from the live filesystem via `FileService`, persisted graph relationships come from `ConnectionService.neighborsByFileLocation`, and tags come from `TagService.tagsByFileLocation`. Before the merged result is returned, every entry is checked with `Files.exists` (and optionally `Files.isHidden`), so

a file deleted between the last sweeper run and the current request is silently dropped rather than surfaced to the user.

A subtle consequence is that files which exist on disk but were never explicitly registered with Graphle are still reachable. Navigating into a directory shows them under the hierarchical descendants even though they have no `File` node yet (F1). When the user acts on one of those entries, the relevant mutation creates the node as a side effect.

5.7.5 Deployment

GraphleManager depends on two external services, Neo4j and Valkey, each with its own lifecycle. `compose.yaml` in the backend directory describes both containers with named volumes (`neo4j_data`, `valkey_data`) so `docker compose up -d` is sufficient to bring up a working development environment. The Gradle `bootRun` task then starts the JVM process itself. The GUI is a separate Gradle project that is either run directly with `./gradlew run` during development or built into a platform-native installer via the Compose Desktop plugin for distribution.

The two services can fail independently, so a Valkey outage while Neo4j remains healthy is an expected operational state. GraphleManager treats it as degraded rather than fatal. Autocomplete stops working, but every other request continues normally (Q3.2).

5.8 Testing

The backend favors integration tests over isolated unit tests. GraphQL resolvers and REST endpoints are exercised through Spring's `MockMvc` against a real Spring context wired up to live Neo4j and Valkey instances, so behavior observed in tests is the same behavior the production process will produce. Pure logic such as DSL token handling, the autocomplete trie, and the concurrent cache is still covered by focused unit tests, but anything that touches persistence is verified end-to-end. Reusable bases (`BaseGraphQlIntegrationTest`, `BaseRestIntegrationTest`) hide the boilerplate of issuing GraphQL operations and HTTP requests, and randomized identifiers keep parallel runs from colliding on shared graph state.

5.8.1 Automated Test Suite

The local test environment requires Neo4j and Valkey. Both services can be started from the backend directory with:

```
cd GraphleManager
docker compose up -d
./gradlew test
```

5.8.2 Autocomplete latency measurement

Autocomplete latency was measured manually from the GUI because the perceived responsiveness of the DSL command line depends on the full round trip over WebSocket, not only on the pure trie lookup. The client timed each autocomplete request from sending the prefix frame to receiving the matching response frame on an already established local `/ws` connection. The samples therefore exclude connection setup and internet latency. Instead, they focus on request handling. This includes Valkey lookup, path reconstruction,

filesystem checks, response transfer, and client-side parsing. Test inputs were randomly chosen path prefixes.

Result count	Samples	Minimum	Average	Median	95th percentile	Maximum
0	21	0.9 ms	1.7 ms	1.5 ms	3.2 ms	3.2 ms
1	29	5.2 ms	15.2 ms	15.8 ms	26.5 ms	28.1 ms
5	40	33.7 ms	47.9 ms	46.6 ms	64.1 ms	66.1 ms

Latency mainly increased with the number of valid paths returned. Prefixes with no result completed almost immediately, while the default five-result case was slower because it had to collect, reconstruct, validate, and encode more candidates. Even the slowest measured group stayed well below the 250 ms limit defined by qualitative requirement Q2.1.

5.8.3 Continuous Integration

The GitHub Actions workflow runs on every push and on pull requests targeting `main`. Neo4j and Valkey are provided as service containers so the same wiring used locally is reproduced in CI. The automatic action ensures the functionality in `main` will not regress once new pull requests with new features start coming in.

Section 6

User Documentation

This chapter provides the end user with a guide to installing, deploying, and using Graphle. It begins with the hardware and software requirements needed to run the application well on supported systems. It then describes the installation and deployment process for the backend services and the desktop client, explains the main user interface screens, and concludes with a tutorial for common Graphle workflows through both the graphical interface and the DSL.

6.1 Hardware and Software Requirements

The hardware and software configuration described below is necessary for Graphle to function optimally. The application is split into four runtime parts: `GraphleManager`, `GraphleUI`, Neo4j, and Valkey. The first two are JVM applications, while Neo4j and Valkey are started by the Docker Compose configuration in `GraphleManager`. The requirements therefore cover both the desktop application resources and the resources consumed by the two background data services.

Hardware requirements:

- **CPU** - The application can put a noticeable load on the processor while actively running. The backend, the desktop UI, Neo4j, and Valkey run as separate processes, so the work can be spread across several cores.
  - Suggestions: A multi-core processor is recommended to run graph queries and all background tasks at the same time without sharing a single core with the GUI.
- **Memory** - Minimum 8 GB RAM. This is sufficient to run all application parts - the JVM backend, the Compose desktop client, the Neo4j container, and the Valkey container, together with a small or medium-sized file collection. 16 GB RAM is recommended. This amount keeps everything running smoothly, because both Neo4j and Valkey keep their data in memory for fast access, and Graphle does the same for the most frequently used autocomplete entries. Bigger file collections therefore need more memory, even though Graphle only keeps information about the files, not the files themselves.
- **Disk space** - The source code itself is small, only a few tens of megabytes. Much more space is taken up by the downloaded dependencies, the Docker images for Neo4j and Valkey, and the stored data, which grows as more files are added.
  - Suggestions: At least 10 GB of free disk space should be reserved for installation and basic use, and 20 GB is recommended to leave room for the stored data to grow. Extra space is also used when the application opens files from a remote `GraphleManager`, because a copy is downloaded first.
- **Internet connection** - Local use does not require an internet connection after installation, but a stable connection with reasonable response times is required when `GraphleUI` connects to a remote `GraphleManager` instance.

Software requirements:

- **Operating system** - The application is intended to run on macOS and Linux.
- **Docker and Docker Compose** - Docker containerization is utilized for the deployment path employed in the next section, so Docker Engine and Docker Compose must be installed. The backend repository contains a `compose.yaml` file that starts Neo4j and Valkey and creates persistent Docker volumes for both services.
- **File permissions** - `GraphleManager` must run under an operating system account that has permission to read and modify the files it manages.

6.2 Installation and Deployment

Graphle is distributed as source code and is intended to be run as a small group of cooperating services. The recommended deployment path uses the provided `start.sh` script, which starts all required parts from the top-level project directory. The script ensures the user does not have to manually start each component in the correct order.

For proper deployment of the application, ensure that the following components are installed in the operating system:

- **JDK 21 or newer** - Required for running both `GraphleManager` and `GraphleUI`.
- **Docker** - Required for the Neo4j and Valkey containers.
- **Docker Compose** - Either the `docker compose` plugin or the older `docker-compose` command can be used.
- **Access to source code** - The source code can be obtained by cloning the GitHub repository with `git clone`.

Initializing the installation with the recommended script consists of the following steps:

1. Move to the project directory, where the `start.sh` script is located.

```
cd Graphle
```

2. Make the startup script executable.

```
chmod +x start.sh
```

3. Run the application by executing the startup script from the top-level project directory.

```
./start.sh
```

The script starts Neo4j and Valkey from `GraphleManager/compose.yaml`, waits until both services accept connections, starts the `GraphleManager` backend with `./gradlew bootRun`, and finally starts the `GraphleUI` desktop client with `./gradlew run`. When the script is interrupted, for example by pressing `Ctrl+C`, it stops the UI and backend processes and shuts down the Docker Compose services.

After a successful start, the desktop application opens on the screen.

The same deployment can also be performed manually. This is useful during development or when one of the components is already running. The manual procedure is:

1. Start the data services from the backend directory.

```
cd GraphleManager
docker compose up -d
```

This starts Neo4j on ports `7687` for the Bolt protocol used by `GraphleManager` and `7474` for the HTTP interface used mainly by Neo4j Browser, and Valkey on port `6379`. Both services store their data in Docker volumes, so the graph database and autocomplete cache survive container restarts.

2. Start the backend service.

```
./gradlew bootRun
```

The backend reads its configuration from `GraphleManager/src/main/resources/application.properties`. By default, it connects to Neo4j at `bolt://localhost:7687`, Valkey at `localhost:6379`, and exposes the application server on port `5824`.

3. Start the desktop client in a second terminal.

```
cd GraphleUI
./gradlew run
```

The client reads its backend configuration from `GraphleUI/src/main/resources/config.yaml`. The `port` value must match the port exposed by `GraphleManager`. The client connects to `localhost` on that port. Remote backends are therefore reached through the SSH port forwarding setup described below. The `localhost` flag describes whether the backend and GUI run on the same machine, which affects how files are opened. When it is set to `false`, opened files are downloaded through `/download` before they are handed to the local operating system.

6.2.1 Remote access over SSH port forwarding

For remote use, the recommended security model is not to expose the `GraphleManager` HTTP port directly to the network. Instead, the administrator starts `GraphleManager`, Neo4j, and Valkey on the remote machine and lets SSH provide the authenticated transport from the user's machine. With the default backend port, the user creates the tunnel from the local machine with:

```
ssh -N -L 5824:127.0.0.1:5824 user@remote-host
```

The first `5824` is the port opened on the local machine. The second `5824` is the `GraphleManager` port on the remote machine. After the tunnel is established, `GraphleUI` still connects to `localhost:5824`, but the traffic is carried by SSH to the remote backend.

For this setup, the GUI configuration should keep the forwarded local port and mark the backend as remote:

```
server:
  port: 5824
  localhost: false
```

The `localhost: false` setting is important because paths returned by the remote backend are meaningful on the remote machine, not on the machine running the GUI. When the user opens a file, Graphle therefore downloads a local copy instead of attempting to open the remote path directly.

Another installation option is to create a native desktop package for `GraphleUI`. This can be done from the UI directory with the following task:

```
cd GraphleUI
./gradlew packageDistributionForCurrentOS
```

The generated package installs only the desktop client. It does not replace the backend, Neo4j, or Valkey deployment, so those services still need to be started with the commands described above.

6.3 User Interface

This section describes the screens and controls of the Graphle desktop application. The graphical client is a single-window application: the header stays visible at the top, while the body changes according to the current display mode. The user can therefore move through the system by opening files, executing DSL commands, opening tags, or using context menus on displayed pills.

6.3.1 Shared Header and Application Menu

Every screen contains the same header. The left side contains the application menu button, and the rest of the header is a command line for DSL commands.

- **Application menu button** - Opens the global menu. The menu contains `Open Home`, `Open Trash`, `Show Hidden Files`, and `Dark mode`. When a file detail screen is active, the same menu also contains actions for the currently opened file.
- **Command line** - Allows the user to enter and execute DSL commands. The command line is also updated when the user navigates through the graphical interface, so it shows the command corresponding to the displayed data. The user can submit the DSL query by pressing `Shift + Enter`.
- **Autocomplete suggestions** - While the user types, the client asks the backend autocomplete service for filename suggestions and displays them below the command line. The user can move through suggestions with the arrow keys or `Tab`, select a suggestion with `Enter`, and execute the current command with `Shift + Enter`.

The file actions available from the application menu and from file context menus are:

- `Open` - Opens the file in the operating system. If the backend is remote, Graphle first downloads the file into the `GraphleDownloads` directory under the user's home directory.
- `Add file` - Creates a new child file. This action is available only for directories.

- `Copy path` - Copies the absolute path to the clipboard.
- `Move` - Opens the move dialog.
- `Add tag` - Opens the add tag dialog.
- `Add relationship` - Opens the add relationship dialog.
- `Remove relationship` - Removes a custom relationship. This action is shown only for relationship pills and not for filesystem parent or descendant links.
- `Move to trash` - Moves the file into the Graphle trash directory.
- `Delete permanently` - Opens a confirmation dialog and deletes the file after confirmation.

6.3.2 File Detail Screen

The file detail screen is the main screen of the application. It is opened after application startup, where it displays the user's home directory, and it is also opened whenever the user selects a file pill or executes a detail command. The same screen is used for normal files, directories, and the Graphle trash directory.

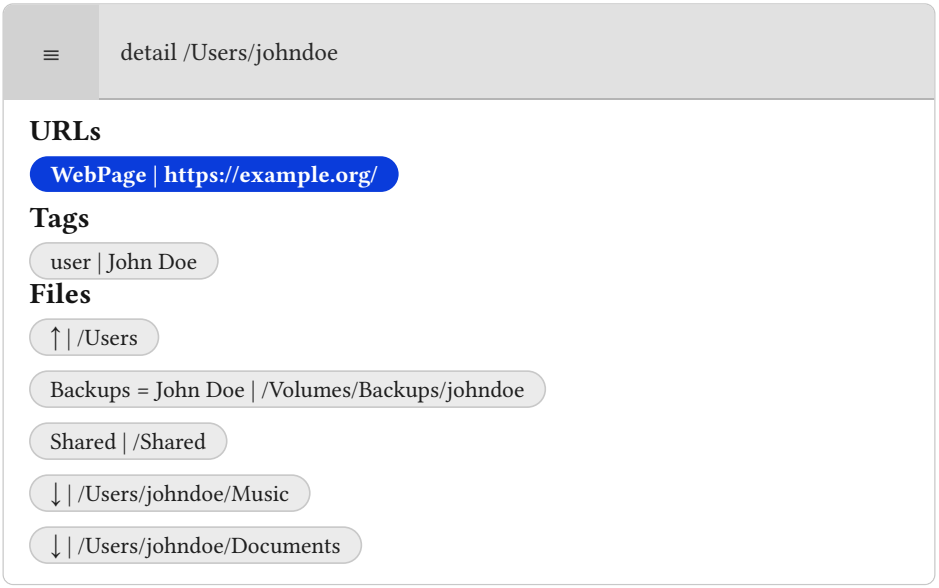


Figure 27: Main page - file detail view

The file detail screen consists of the following parts:

- **Header** - Contains the application menu and the DSL command line. In this screen, the command line corresponds to a `detail` command for the currently displayed path.
- **URLs section** - Shows tags whose value is a valid URL. Clicking a URL tag opens the URL. Opening the context menu on the tag allows the user to search for files with the same tag name or delete the tag from the current file.
- **Tags section** - Shows normal tags attached to the current file. Each tag is displayed as a pill containing the tag name and, if present, the tag value. The tag context menu contains `Open`, which displays all files carrying the tag name, and `Delete`, which removes the tag from the current file.
- **Files section** - Shows related files. Filesystem parent and descendant links are displayed together with custom graph relationships. Parent links use an upward arrow, descen-

dant links use a downward arrow, and custom relationships show their relationship name and optional value. Clicking a file pill opens that file in the detail screen. Opening the file pill context menu gives access to the file actions described above.

6.3.3 Filename Results Screen

The filename results screen is displayed when a DSL command returns a list of file paths. This typically happens after a `find` command that ends with a file scope, for example a search by tag name, tag value, location, or relationship traversal followed by an empty file scope. Scopes and the DSL are explained further in the following section.

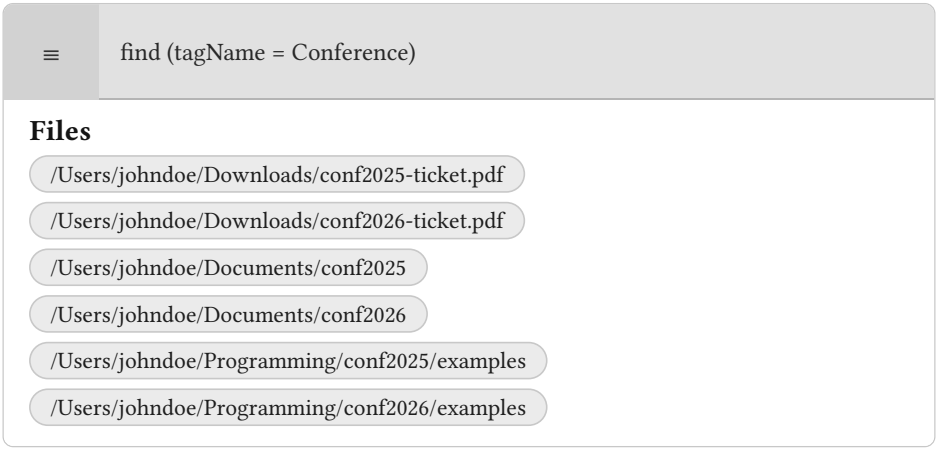


Figure 28: Main page - filenames view

The screen contains:

- **Header** - Shows the command that produced the results.
- **Files section** - Lists all returned file paths as pills. Clicking a pill opens the selected file in the file detail screen. Opening the context menu on a pill gives access to the same file actions as in the file detail screen.

6.3.4 Relationship Results Screen

The relationship results screen is displayed when a `find` command ends with a relationship scope and therefore returns relationships rather than target files. It has the same visual structure as the file list, but each pill contains both relationship information and the related file path.

Figure 29: Main page - find results with relationship filter view

The screen contains:

- **Header** - Shows the executed DSL command.
- **Files section** - Lists returned relationships. Each pill includes the relationship name, optional relationship value, and target file path. Clicking a pill opens the target file. Opening the context menu gives access to file actions and, for custom relationships, it also gives the `Remove relationship` action.

6.3.5 Files With Tag Screen

The files with tag screen is displayed after opening a tag from the tag context menu or after executing the `tag` DSL command. It lists all files that carry the requested tag name.

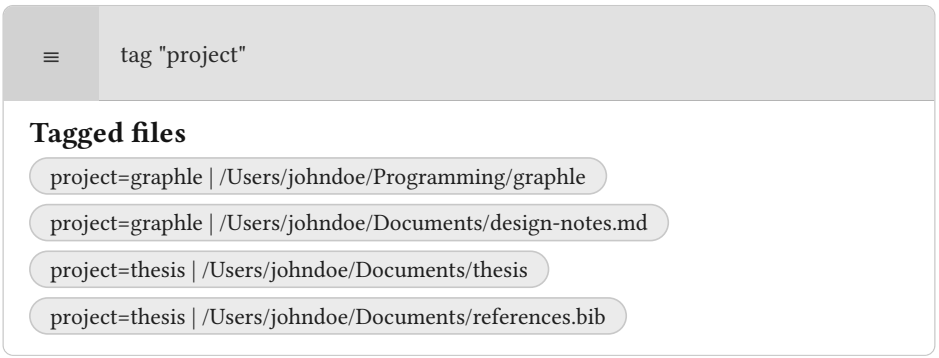


Figure 30: Main page - files with tag view

The screen contains:

- **Header** - Shows the tag lookup command.
- **Tagged file list** - Displays one pill for each matching file. The pill contains the tag name, the optional tag value, and the file path. Clicking a pill opens the corresponding file in the file detail screen.

6.3.6 Dialog Screens

Several operations are performed through dialogs.

6.3.6.1 Add Tag Dialog

The add tag dialog is opened by selecting the `Add tag` file action. It contains:

- `Tag name*` - Required field containing the tag name.
- `Tag value` - Optional field containing the tag value.
- `OK` - Creates the tag and refreshes the current file detail screen.
- `Cancel` - Closes the dialog.

6.3.6.2 Add Relationship Dialog

The add relationship dialog is opened when the `Add relationship` file action is invoked. It creates a relationship from the current file to another file. It contains:

- `Related to*` - Required path of the target file.
- `Relationship name*` - Required relationship name.
- `Relationship value` - Optional relationship value.
- `Is bidirectional` - Checkbox that creates the relationship in both directions when selected.

- `OK` - Creates the relationship and refreshes the current file detail screen.
- `Cancel` - Closes the dialog.

6.3.6.3 Add File Dialog

The add file dialog is shown after selecting the `Add file` action on a directory. It contains:

- `Filename` - Required name of the new child file.
- `OK` - Creates the file under the current directory and refreshes the screen.
- `Cancel` - Closes the dialog.

6.3.6.4 Move File Dialog

The move file dialog is shown after selecting the `Move` file action. It contains:

- `Move to` - Required path of the destination directory.
- `OK` - Moves the current file into the selected directory.
- `Cancel` - Closes the dialog.

6.3.6.5 Delete File Dialog

The delete file dialog is opened from the `Delete permanently` file action. It asks the user to confirm permanent deletion of the selected path. The dialog contains `OK`, which permanently deletes the file, and `Cancel`, which closes the dialog.

6.4 Tutorial

6.4.1 DSL

This section explains how to use the provided DSL interface. The DSL provides the same graph operations available from the graphical user interface, plus more. This includes inspecting files, adding tags, creating relationships, and searching through the graph. It is useful when the user wants to integrate the Graphle system into their own scripts, or express a search that would be inconvenient to build through the GUI.

6.4.1.1 Running a command

In the desktop application, commands are entered into the command line in the header. The same commands can also be sent directly to the backend via HTTP by posting to `/dsl`. For example, if `GraphleManager` is running on the default port, the following command asks for all files carrying the `project` tag:

```
curl \
  -H 'Content-Type: application/json' \
  -X POST \
  -d '{"command":"tag \"project\""}' \
  http://localhost:5824/dsl
```

The server returns a `DSLResponse`. Successful upsert commands return the type `SUCCESS`. Search commands return one of `FILENAMES`, `CONNECTIONS`, `FILE`, or `TAG`, depending on what was requested. If the command cannot be parsed or executed, the response type is `ERROR`.

6.4.1.2 Basic commands

The simplest commands operate on one file, one tag, or one relationship. Names, values, and paths can be written without double quotes, but multi-word entries should use double quotes.

Command	Effect
`detail "<path>"`	Shows one file together with its tags and relationships.
`tag "<name>"`	Lists files that carry a tag with the given name, regardless of the tag value.
`addTag "<path>" "<name>"`	Adds a simple tag to a file.
`addTag "<path>" "<name>" "<value>"`	Adds a key-value tag to a file.
`removeTag "<path>" "<name>"`	Removes a simple tag from a file.
`removeTag "<path>" "<name>" "<value>"`	Removes a key-value tag from a file.
`addRel "<from>" "<to>" "<name>"`	Creates a directed relationship from one file to another.
`addRel "<from>" "<to>" "<name>" "<value>"`	Creates a directed relationship with an additional value.
`removeRel "<from>" "<to>" "<name>"`	Removes a directed relationship without a value.
`removeRel "<from>" "<to>" "<name>" "<value>"`	Removes a directed relationship with the given value.
`addFile "<path>"`	Creates a file at the given path and registers it for completion.
`removeFile "<path>"`	Removes a file or directory and deletes its graph metadata.
`moveFile "<from>" "<to>"`	Moves a file and updates the stored graph location.

For example, the following commands tag a film with its type and release year:

```
addTag "/Users/johndoe/GraphleDslSample/movies/the-matrix.mkv" "type"
"movie"
addTag "/Users/johndoe/GraphleDslSample/movies/the-matrix.mkv" "year"
"1999"
```

The next command links a research note to the plan it motivates:

```
addRel "/Users/johndoe/GraphleDslSample/research/notes/research-
questions.md" "/Users/johndoe/GraphleDslSample/projects/graphle-demo/
experiment-plan.md" "motivates"
```

6.4.1.3 Finding files

The `find` command is the expressive part of the DSL. It is built from scopes:

- file scopes use parentheses: `( ... )`
- relationship scopes use square brackets: `[ ... ]`

A file scope filters file nodes. It can use the fields `location`, `tagName`, and `tagValue`. A relationship scope filters edges between the currently selected files and their neighbors. It can use the fields `name` and `value`.

The supported operators are `=`, `!=`, `<>`, `>`, `>=`, `<`, and `<=`. Predicates can be combined with `AND` and `OR`, and parentheses can be used inside a scope to make precedence explicit. Numeric comparisons are most useful with numeric tag values, such as years:

```
find (tagName = "year" AND tagValue >= 2000)
```

This returns files whose `year` tag has a numeric value greater than or equal to `2000`.

To find all files tagged as movies or shows, use:

```
find (tagName = type AND (tagValue = show OR tagValue = movie))
```

To find one file by its path, use:

```
find (location = "/Users/johndoe/GraphleDslSample/research/notes/research-
questions.md")
```

If no filter is given for the first file scope, Graphle displays all indexed files to avoid having to fetch every file present on disk.

6.4.1.4 Traversing relationships

Scopes are evaluated from left to right. A relationship scope uses the files selected by the previous file scope as its starting points. If a command ends with a relationship scope, the result is a list of relationships. If the user wants the target filenames instead, an empty file scope `()` can be appended.

The following command starts at `research-questions.md`, follows outgoing relationships named `motivates`, and returns the target files:

```
find (location = "/Users/johndoe/GraphleDslSample/research/notes/research-
questions.md")[name = "motivates"]()
```

Longer traversals are written by alternating relationship and file scopes. For example, this command follows a `motivates` relationship and then a `produces` relationship:

```
find (location = "/Users/johndoe/GraphleDslSample/research/notes/research-
questions.md")[name = "motivates"]()[name = "produces"]()
```

The special relationship scopes `[DESC]` and `[PRED]` expose the live filesystem hierarchy. `[DESC]` selects direct descendants of a directory, while `[PRED]` selects the parent directory of the current file. For example, the following command lists the direct children of the sample movie directory:

```
find (location = "/Users/johndoe/GraphleDslSample/movies")[DESC]()
```

Because these hierarchy relationships are read from the filesystem, they can be used even when the files are not yet registered in Graphle.

6.4.1.5 Suggested workflow

A typical scripted workflow has two phases. First, create or collect the files with normal filesystem tools such as `mkdir` and `touch`. Second, initialize the graph metadata with `addTag` and `addRel`. After that, day-to-day work can use `find`, `tag`, and `detail` to query the graph.

For a small research project, a user might tag notes, datasets, and generated summaries:

```
addTag "/Users/johndoe/GraphleDslSample/research/notes/literature-
review.md" "type" "note"
addTag "/Users/johndoe/GraphleDslSample/research/datasets/interviews.csv"
"type" "dataset"
addTag "/Users/johndoe/GraphleDslSample/projects/graphle-demo/results-
summary.md" "type" "summary"
addRel "/Users/johndoe/GraphleDslSample/research/datasets/interviews.csv"
"/Users/johndoe/GraphleDslSample/projects/graphle-demo/results-summary.md"
"input-for"
```

The graph can then be queried by category:

```
find (tagName = "type" AND tagValue = "dataset")
```

or by semantic connection:

```
find (location = "/Users/johndoe/GraphleDslSample/research/datasets/
interviews.csv")[name = "input-for"]()
```

6.4.1.6 Demo setup

The repository contains a small runnable example for this workflow in the `examples` directory. The script `examples/graphle-dsl-home-setup.sh` creates a sample hierarchy under `~/GraphleDslSample`, creates empty files, and then initializes Graphle metadata through the DSL. The generated hierarchy contains research notes, a dataset, project files, an archive file, and a movie collection.

The same script tags the files with values such as `type = movie` or `year = 1999`. It also creates relationships such as `motivates` or `input-for`.

After the sample has been initialized, query files can be executed by passing a DSL file to `examples/run-graphle-dsl-file.sh`.

```
examples/run-graphle-dsl-file.sh examples/queries/04-movies-by-type.dsl
```

The `examples/queries` directory contains separate query files to test out the system:

- `01-project-tags.dsl` lists files with the `project` tag.
- `02-year-tags.dsl` lists files with the `year` tag.
- `03-motivated-plan.dsl` follows the `motivates` relationship from the research questions note.
- `04-movies-by-type.dsl` finds files tagged as movies.
- `05-movies-from-2000.dsl` finds files whose numeric `year` tag is at least `2000`.
- `06-movie-directory-children.dsl` uses `[DESC]` to list the direct filesystem children of the movie directory.

6.4.2 GUI

The graphical interface provides the same core operations as the DSL, but organizes them around the currently displayed file or directory. When the application starts, it opens the user's home directory and shows its tags and connected files. The top row contains the application menu on the left and the command line next to it. The command line can still be used for DSL commands, but most common operations are available from menus and dialogs.

6.4.2.1 Main view

The main body of the application is a detail view for the selected file or directory. It is divided into three kinds of information:

- URL tags, shown separately when a tag value is a valid URL.
- Other tags, containing the tag name and optional value.
- Related files, containing the relationship name and the target file path.

Clicking a file opens that file's detail view. Parent and descendant relationships are displayed together with custom relationships, so the user can navigate both the normal filesystem hierarchy and manually created graph relationships from the same screen. Parent relationships are displayed with an upward arrow and descendant relationships with a downward arrow.

When a command or menu action returns a list of filenames rather than a single file, the application shows a list of filename pills. Clicking one of these filenames opens the corresponding detail view. When a tag lookup is opened, the application shows files carrying that tag together with the matched tag value.

6.4.2.2 Application menu

The application menu is opened with the button on the left side of the header. It contains global actions and, when a file detail view is active, also file actions for the selected path.

The main global actions are:

- `Open Home` returns the user's home directory.
- `Open Trash` opens the trash view.
- `Show Hidden Files` toggles whether hidden filesystem entries are displayed.
- `Dark mode` toggles the visual theme.

When the current view is a file or directory detail view, the same menu also exposes the file actions described below.

6.4.2.3 File context menu

File and relationship pills can be opened directly with a click. Opening the context menu on a file pill gives access to operations for that file:

- `Open` opens the file with the default app. If the backend is remote, Graphle first downloads a copy into `GraphleDownloads` under the user's home directory.
- `Add file` is available on directories and creates a new child file.
- `Copy path` copies the absolute path to the clipboard.
- `Move` moves the file to another directory.
- `Add tag` opens a dialog for entering a tag name and optional value.
- `Add relationship` opens a dialog for linking the current file to another file.
- `Remove relationship` is shown for custom relationships and removes that graph edge.
- `Move to trash` moves the file to the trash.
- `Delete permanently` removes the file after confirmation.

Parent and descendant relationships come from the live filesystem, so they cannot be removed as custom Graphle relationships. Only manually created relationships have the `Remove relationship` action.

6.4.2.4 Adding tags

To add a tag through the GUI, open the menu for the target file and choose `Add tag`. The dialog requires a tag name and accepts an optional value.

Existing tags are displayed as pills in the detail view. Opening a tag pill's menu provides two actions. `Open` shows all files that carry a tag with the same name. `Delete` removes that tag from the current file. If a tag value is a URL, clicking the tag opens the URL directly.

6.4.2.5 Adding relationships

To create a relationship, open the source file's menu and choose `Add relationship`. The dialog requires:

- `Related to`, the absolute path of the target file.
- `Relationship name`, the semantic name of the edge.

The optional `Relationship value` field can store extra information about the relationship. The `Is bidirectional` checkbox creates the relationship in both directions, which is useful when the connection should be navigable from either file.

Clicking the relationship pill navigates to the target file. Opening the pill's context menu allows the user to remove the relationship.

6.4.2.6 Searching from the GUI

The GUI can search in two ways. First, the user can browse interactively by clicking file, relationship, and tag pills. Second, the command line in the header accepts the same DSL commands described in the previous section. While typing, the application asks the backend for autocomplete suggestions. Suggestions can be selected from the list, and the completed command can then be executed from the command line. Once the command is complete, press Shift + Enter to submit it, and the backend response will be rendered in the browser.

Section 7

Conclusion

This thesis set out to design and implement Graphle, a graph-oriented file management system that lets users organize files by semantic association while being backward-compatible with the standard filesystem. Graphle keeps files in ordinary folders and adds a graph layer for tags and typed relationships. This lets users search and move between files by meaning, not only by path, without replacing the filesystem.

The main contribution is a working design in which files and folders are represented as nodes in a LPG, while file contents, operating-system metadata, and the native hierarchy stay on disk. Persisted graph data is limited to the semantic information added by the user. Parent and descendant relationships are derived from the live filesystem when needed, which keeps Graphle compatible with existing tools and avoids duplicating data already maintained by the operating system. This model gives each file a stable place in the hierarchy while still allowing it to participate in many independent associations.

Graphle supports two ways of working with files. The GUI lets users browse the filesystem, open, move, and delete files, and edit tags and relationships in one place. The custom DSL provides a command-based alternative for users who need more complex queries.

The system addresses consistency between the graph and the live filesystem by combining lazy loading with background maintenance. Files are loaded into the graph only when they are visited or queried, so startup does not require a full disk scan. A background sweeper removes graph entries for files that have disappeared from the underlying filesystem, preventing stale relationships and tags from accumulating after external changes. Together, these mechanisms keep Graphle usable alongside ordinary filesystem tools.

In conclusion, Graphle shows that files can stay in the existing filesystem while being organized and explored as a graph. This thesis provides the data model, architecture, query language, user interface, and implementation for that approach. It also creates a base for better clients and for testing how useful graph-based file organization is in practice.

The most important future extension is application-level authentication and authorization for remote access. The current implementation supports remote use through SSH port forwarding and relies on operating-system accounts and filesystem permissions. A more broadly deployed version should add a uniform authentication layer across the GraphQL, REST, and WebSocket interfaces so that the backend can protect the exposed filesystem even when it is reachable by remote clients.

Bibliography

[1] Vinayak Baranwal, " SSH Port Forwarding: Local, Remote, and Dynamic Explained ." [Online]. Available: https://www.digitalocean.com/community/tutorials/ssh-port-forwarding

[2] " SSH configuration | DBeaver Documentation ." [Online]. Available: https://dbeaver.com/docs/dbeaver/SSH-Configuration/

[3] Apple Inc., "Finder – Mac Help." [Online]. Available: https://support.apple.com/guide/mac-help/finder-mchlp2605/mac

[4] The GNOME Project, "GNOME Files (Nautilus)." [Online]. Available: https://apps.gnome.org/Nautilus/

[5] KDE, "Dolphin File Manager." [Online]. Available: https://apps.kde.org/dolphin/

[6] KDE, "Dolphin – Embedded Terminal." [Online]. Available: https://docs.kde.org/stable5/en/dolphin/dolphin/dolphin-terminal.html

[7] Obsidian, "Obsidian - Sharpen your thinking." [Online]. Available: https://obsidian.md/

[8] Michael Brenan, "Dataview – Obsidian Plugin." [Online]. Available: https://blacksmithgu.github.io/obsidian-dataview/

[9] TagSpaces, "Organize your files and folders with tags | TagSpaces." [Online]. Available: https://www.tagspaces.org/

[10] Paul Ruane, "TMSU." [Online]. Available: https://tmsu.org/

[11] William Ting, "autojump – A cd Command That Learns." [Online]. Available: https://github.com/wting/autojump

[12] Oracle, "Java." [Online]. Available: https://www.java.com/

[13] JetBrains, "Kotlin Programming Language." [Online]. Available: https://kotlinlang.org/

[14] VMware, "Spring Boot." [Online]. Available: https://spring.io/projects/spring-boot

[15] The GraphQL Foundation, "GraphQL: A Query Language for Your API." [Online]. Available: https://graphql.org/

[16] JetBrains, "Compose Multiplatform." [Online]. Available: https://www.jetbrains.com/lp/compose-multiplatform/

[17] Neo4j, Inc., "RDF vs. Property Graphs in Knowledge Graphs." [Online]. Available: https://neo4j.com/blog/knowledge-graph/rdf-vs-property-graphs-knowledge-graphs/

[18] ArangoDB Inc., "ArangoDB Graphs Documentation." [Online]. Available: https://docs.arango.ai/arangodb/stable/graphs/

[19] Arcade Data Ltd, "ArcadeDB: Open-Source Multi-Model Graph Database." [Online]. Available: https://arcadedb.com/

[20] Neo4j, Inc., "Neo4j Graph Database." [Online]. Available: https://neo4j.com/

[21] Stack Overflow, "Stack Overflow Developer Survey 2024: Most Popular Technologies – Database." [Online]. Available: https://survey.stackoverflow.co/2024/technology#most-popular-technologies-database

[22] Broadcom, "Spring Data Neo4j." [Online]. Available: https://spring.io/projects/spring-data-neo4j/

[23] Human Benchmark, "Reaction Time Statistics." [Online]. Available: https://humanbenchmark.com/tests/reactiontime/statistics

[24] The Linux Foundation, "Valkey – An Open Source, In-Memory Data Store." [Online]. Available: https://valkey.io/

[25] Redis Ltd., "Redis." [Online]. Available: https://redis.io/

[26] Redisson, "Valkey vs Redis: Performance Comparison." [Online]. Available: https://redisson.pro/blog/valkey-vs-redis-comparision.html

[27] Edward Targett, "Redis Fork Valkey Joins the Linux Foundation." [Online]. Available: https://www.thestack.technology/redis-fork-valkey-linux-foundation/

Appendix A

Vocabulary

API (Application Programming Interface) - A public interface exposed by a software component so other components can call its functionality

Apollo - A GraphQL client library used by GraphleUI to generate and execute typed GraphQL operations

Cache - A faster temporary storage layer used to avoid repeatedly reading the same data from a slower source

Connection - An edge between entities in a database

Relationship - Logical connection between entities describing how they are conceptually related

Neighbor - Entities connected via a direct connection

Tag - Category of an entity

DSL (domain-specific language) - Domain-specific language

Filesystem - The operating system structure that stores files and directories and controls access to them

GraphQL - A query language for APIs that allows clients to request exactly the data they need

HTTP (Hypertext Transfer Protocol) - The request-response protocol used by web APIs and browsers

JSON (JavaScript Object Notation) - A text format for structured data commonly used in API requests and responses

LAN (Local Area Network) - A private network connecting devices in a limited area, such as a home or office

Lazy loading - Loading data only when it is requested instead of eagerly loading everything in advance

LPG (Labeled Property Graph) - A graph data model in which nodes and edges carry both labels and arbitrary key-value properties

Metadata - Data that describes another object, such as tags or relationships describing a file

REST (Representational State Transfer) - An API style that exposes operations through HTTP resources and methods

SFTP (SSH File Transfer Protocol) - A protocol for transferring files over an encrypted SSH connection

SMB (Server Message Block) - A network file sharing protocol commonly used for accessing shared folders

Symbolic link - A filesystem entry that points to another file or directory path

WebSocket - A protocol for a persistent bidirectional connection between a client and a server

Cypher - The declarative query language used by Neo4j to express graph patterns

Trie - A tree data structure where each node represents a character, used for efficient prefix-based lookups